

Graduado en Ingeniería Informática

Universidad Politécnica de Madrid
Escuela Técnica Superior de
Ingenieros Informáticos

TRABAJO FIN DE GRADO

Automatización de despliegue de instancias de openstack con ansible

Autor: Alejandro Moreno Mateos
Director: Ángel Rodríguez de Bartolomé

MADRID, JUNIO 2016

RESUMEN

La meta de este trabajo es conseguir montar un sistema *cloud* privado y *opensource* para poder lanzar instancias virtuales y trabajar con ellas. Esto se debería llevar a cabo sobre computadores que proporcionasen la funcionalidad de un servidor de servicios, pero como no se pudo hacer uso de estos, se utilizó un software de virtualización, en este caso `virtualbox`[4].

Tras un estudio previo en el que se escogió la mejor arquitectura posible teniendo en cuenta aspectos de funcionalidad y seguridad se lleva a cabo el proyecto sobre tres máquinas virtuales. Había dudas sobre la viabilidad del proyecto en la fase de diseño porque el producto final es virtualización sobre virtualización, pero no había otro modo de poder llevarlo a cabo. Para la parte de automatización se emplea `ansible`[8] de manera que con la ejecución de un comando se lleve a cabo todo lo necesario para que `openstack`[2, 3] esté disponible y se pueda lanzar instancias con él. Este mandato hará bastantes cosas internamente para conseguir la meta. En primer lugar ha de crear las máquinas virtuales sobre las que se montará `openstack` y configurar su almacenamiento, memoria, red...

Una vez hecho esto debería configurar la máquina de NAT para tener una configuración homogénea que daremos a todas las máquinas. Posteriormente configurará el `controller`. Éste, lo primero que hará será configurar el entorno de la máquina y después seguirá con los servicios necesarios de `openstack`. Aquí se instalará y configurará `keystone` para la autenticación, `glance` para el servicio de imágenes, `nova` para el servicio de computación, `neutron` para la red y `horizon` como `dashboard`. Por último se encargará de hacer lo mismo en la máquina de cómputo pero con algún servicio menos ya que no van a ser necesarios para la computación. Teniendo esto ya debería ser capaz de acceder vía web al sistema e interactuar con él. Además, alternativamente se puede incluir que lance una instancia automáticamente por si el usuario es inexperto.

Tras las pruebas del sistema el resultado es satisfactorio y cumple con la funcionalidad inicialmente prevista.

ABSTRACT

As I have said in the introduction section the goal of this project is to build a private and open-source cloud system for launching instances and distribute jobs within them. To accomplish this goal, I would need some physical servers but I haven't them. Then I'm going to use `virtualbox` software building `openstack` inside it.

Before the previous study I'm going to use three `virtualbox` instances for this project for security and functional reasons. I wasn't sure that the infrastructure will work over `virtualbox` machines because this is virtualization over virtualization. I was a little scared for this but it looks that is going to work correctly. For the automatization I'm going to use `ansible` because I want that when I execute a command I need to assure that all the `openstack` infrastructure is mounted and the instance can be launched. The command has to perform a lot of things. The first thing is to create and configure the `virtualbox` machines (create, memory, network...). Then, it must configure the NAT virtual machine with a standard configuration that I have created for all machines.

The most important machine is the controller. I have to install and configure `keystone` (in it for authentication), `glance`(image service), `nova` (compute service), `neutron` (networking service) and `horizon` (dashboard service). Last, the command will make the same operation in the compute node but with some less services. With all these things I could launch the `openstack` instances in a web. Also I can create a playbook for automatically launching new instances if the user is inexperienced.

With the system testing I can see a great results for the project.

Índice de contenido

CAPÍTULO 1: INTRODUCCIÓN Y OBJETIVOS.....	1
CAPÍTULO 2: DISEÑO DEL SISTEMA.....	3
2.1- Creación de la infraestructura cloud.....	3
2.2- Redes.....	5
2.3- Hardware virtualizado.....	7
2.4- Automatización.....	9
2.5- Control de versiones.....	11
2.6- Openstack.....	12
CAPÍTULO 3: IMPLEMENTACIÓN.....	15
3.1- NAT.....	15
3.2- Controller.....	17
3.2.1- Entorno.....	19
3.2.2- Identity Service (Keystone).....	21
3.2.3- Image Service (Glance).....	25
3.2.4- Compute Service (Nova).....	28
3.2.5- Networking Service (Neutron).....	32
3.2.6- Dashboard Service (Horizon).....	34
3.3- Compute.....	36
3.3.1- Entorno.....	37
3.3.2- Compute Service (nova).....	38
3.3.3- Networking Service (neutron).....	39
CAPÍTULO 4: PRUEBAS DEL SISTEMA.....	40
CAPÍTULO 5: RESULTADOS Y CONCLUSIONES.....	48
BIBLIOGRAFÍA.....	50
ANEXO 1: ACRÓNIMOS.....	51
ANEXO 2: GLOSARIO.....	52

Índice de figuras

Figura 1. Tipos de nodos necesarios para una infraestructura de openstack.....	3
Figura 2. Representación de las redes pública y privada.....	4
Figura 3. Redes presentes en la máquina NAT.....	5
Figura 4. Redes presentes en las máquinas Controller y Compute.....	6
Figura 5. Representación de los adaptadores de red.....	7
Figura 6: Logo del orquestador Ansible.....	9
Figura 7. Gitflow. La rama máster se ha representado en color verde. El resto de nodos pertenecen a diferentes ramas sobre las que se han ido realizando sucesivos cambios..	12
Figura 8. Logo representativo del proyecto openstack.....	13
Figura 9. Capa de servicios de openstack.....	14
Figura 10. Configuración de red de un sistema con máquina NAT.....	15
Figura 11. Template del host.....	16
Figura 12. Módulo wait_for Ansible.....	18
Figura 13. Sincronización NTP.....	19
Figura 14. Logo representativo de mongoDB.....	20
Figura 15. Logo representativo de RabbitMQ.....	20
Figura 16. Flujo Keystone. Interacción de usuarios con API.....	21
Figura 17. Creación de base de datos con Ansible.....	22
Figura 18. Glance flow. Petición mediante la API empleando keystone para autenticar.	26
Figura 19: Aprovisionamiento de una instancia con autenticación mediante keystone, imagen provisionada por glance y redes por neutron.....	30
Figura 20. Dashboard login.....	35
Figura 21: Vista general del proyecto.....	35
Figura 22: Token generado por keystone.....	40
Figura 23: Imagen generada por glance.....	41
Figura 24: Servicio de cómputo en funcionamiento.....	42
Figura 25: Endpoints de las APIs de los servicios.....	43
Figura 26: Estado de los agentes de neutron.....	44
Figura 27: login de openstack.....	44
Figura 28: Apartado de instancias del dashboard.....	45
Figura 29: Configuración de una nueva instancia.....	46
Figura 30: Configuración de redes de la instancia.....	47
Figura 31: Instancia creada.....	47

CAPÍTULO 1: INTRODUCCIÓN Y OBJETIVOS

Hoy en día estamos sumidos en un mundo en el que no hay un estándar de tecnologías, sino que hay muchas, diversas y todas son utilizadas en mayor o menor medida. Estas tecnologías se emplean para construir aplicaciones, paginas web, servicios... Pero todo esto ¿cómo funciona? ¿Dónde se encuentra? Alrededor de estas respuestas siempre van a estar los sistemas servidores de proceso. Sí, esos hierros albergados en *racks* dentro de los CPD. Esos que hay que dar cariño cada vez que un disco se rompe o reiniciar si no responden... Es un mundo, que hoy en día, creo que cada vez está reservado a unos pocos privilegiados, o no tanto... según opiniones. Si interesa la mía creo que finalmente estos estarán tan centralizados en determinados sitios que dudaremos de su existencia. Eso sí, allá donde estén serán una infinidad de ellos.

¿Porqué estas reflexiones rondan mi cabeza? Todo ello es culpa de ese gran desconocido, el *cloud*. Aún hoy lo es para mucha gente pero está ocurriendo una gran transición entre el CPD convencional y el *cloud*. Además todo lo anterior hay que aderezarlo con todos los términos que han surgido para los servicios (IAAS, SAAS, PASS). Por añadidura a todo este maremágnum, el *cloud* puede ofrecerse de varias formas.

Entonces, todo esto del *cloud*, ¿qué es? El *cloud* al final acaba siendo un conjunto de servicios corriendo en máquinas de CPD, como comentaba al principio, configuradas para crear esta “ilusión” gracias al poder de la virtualización. Por ello los CPD no se pueden extinguir. Es cierto que es mucho más complicado tener máquinas físicas y tener que administrarlas que tenerlas en *cloud*. Esto es porque sabes que alguien va a velar por la salud de la máquina física donde corren tus nodos virtualizados. Por ello la tendencia que se está dando últimamente es ir hacia esa despreocupación.

Pero, ¿por qué entonces el número de CPD será mas reducido y voluminoso? Eso es sencillo. Al tender hacia el *cloud* las máquinas de las empresas dejarán de estar albergadas en los CPD y comenzarán a estar en él. Por ello no todo el mundo podrá ofrecer los mismos servicios. Habrá empresas mucho más experimentadas que ofrezcan sus servicios *cloud* con una mayor calidad y por ello tendrá muchos mas clientes con las consiguientes ventajas. Hoy en día Amazon parece que tiene el terreno ganado pero ahí vienen otros dos gigantes como Azure y Google pisando los talones para disputarse la victoria en estos ámbitos. Todo esto es intrigante. ¿Qué correrán esas multinacionales en sus máquinas para crear la ilusión del *cloud*? ¿Utilizarán todos lo mismo? ¿Tendrán desarrollos privados? ¿Usarán tecnologías *opensource*? ¿O será un conjunto de todo esto? Creo que esto nunca lo sabremos porque será de los secretos mejor guardados de estas empresas.

Entonces ¿cuál es la forma en que podemos ver cómo funcionan los entresijos de un

cloud? Ahí viene el protagonista de este trabajo, *openstack*. Esta tecnología combina varios proyectos llevados a cabo por reconocidas empresas del sector IT cuya finalidad es levantar un *cloud* privado y *opensource* el cual se puede montar modularmente según las necesidades que se necesiten. No hay un montaje concreto sino que puedes añadir los módulos que sean necesarios. Todo esto nos permite jugar a ser alguna de las empresas antes mencionadas. Una vez montado todo este sistema podemos entrar a través de una interfaz web a un *dashboard* en el que levantar instancias sin preocuparnos por nada más. Uno de los puntos fuertes hoy en día de las máquinas en *cloud* es el pago por uso. Si tú levantas una instancia dos minutos sabes que sólo vas a pagar esos dos minutos. Eso le interesa a la gran mayoría de la gente.

Entonces, ¿qué persigo con este proyecto? Fácil: ser capaz de montar mi propio *cloud* privado en el que levantar instancias. Pero esto es un proceso complejo. Las configuraciones de *openstack*[1] no son sencillas y además hay que entenderlo primero. No sería útil tardar un mes cada vez que hubiese que replicar la infraestructura... Pues la solución que se me ocurre es automatizar el proceso para que en cuestión de minutos se pueda hacer. Y qué mejor para hacerlo que un orquestador* que es propiedad de una de las empresas que colaboraron en la creación de *openstack*, Redhat. Este orquestador es *ansible*, que cada vez se va abriendo más camino en este mundo. Creo que la idea es buena y posiblemente factible. Veremos a continuación los problemas que preveo que me encontraré.

El contenido de la memoria se ha estructurado en un primer capítulo de introducción y objetivos donde se acaba de describir el origen de este TFG y el objetivo del mismo. A continuación se expondrá la fase de diseño de infraestructura, redes, hardware. Posteriormente se tratará la fase de implementación, diferente en cada uno de los nodos de la infraestructura así como la fase de pruebas del sistema. Por último expondré los resultados y las conclusiones obtenidas a lo largo de este TFG así como la bibliografía y anexos que recojan elementos adicionales.

*Orquestador: Anexo 2

CAPÍTULO 2: DISEÑO DEL SISTEMA

2.1- Creación de la infraestructura cloud

Este apartado fue una de las primeras cosas que tuve que abordar para llevar a cabo el proyecto. Como tenía que montar la infraestructura virtualizada en mi portátil (limitado por 8GB de memoria RAM) en principio pensé en montarlo en una sola máquina virtual.

Esto no me convenció por el motivo de que en un entorno productivo, las máquinas de cada uno de los nodos de `openstack` pueden ser múltiples y quería hacerlo lo mas parecido a mi idea de un entorno que ya está en producción. Openstack consta de dos partes diferenciadas, una de control y otra de computación. Ambas pueden ser configuradas en un mismo servidor aunque lo más común es poder aislar uno de otro y que además haya más de una máquina para cada tipo.

Soy consciente de que no podía dividir la infraestructura en múltiples nodos por las limitaciones de mi hardware, así que decidí intentar separar el nodo de control del nodo computacional. Así tendría una estructura “limpia y ordenada” en la que se separarían las máquinas según su funcionalidad tal y como se muestra en la Figura 1.



Figura 1. Tipos de nodos necesarios para una infraestructura de openstack.

Esto está muy bien para mi máquina, en la que se que sólo yo tengo acceso. Pero, ¿y si esto lo ponemos en máquinas físicas? Creo que es necesario incorporar mecanismos de seguridad en su acceso.

La idea que tengo en este punto es dejar los nodos de computación y de control en una subred privada. Para ello pondré una máquina de NAT delante de ambos nodos. Las

máquinas llegaran a la red externa a través de ella y se podrá filtrar mediante su *firewall* las peticiones. De esta forma nos queda lo indicado en la Figura 2.

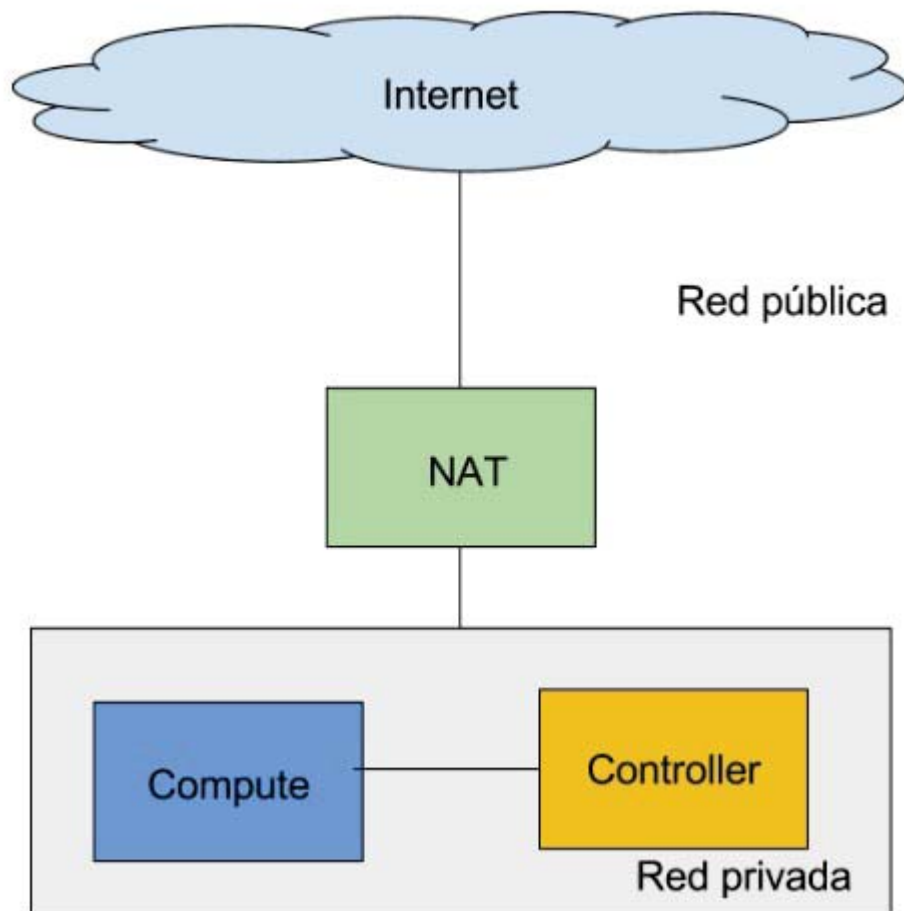


Figura 2. Representación de las redes pública y privada.

Ahora sí creo que es la infraestructura correcta para poder llevar a cabo el montaje de openstack.

2.2- Redes

Para este entorno es requisito que las máquinas estén en una misma red de manera que puedan comunicarse entre sí y tengan acceso a una red. También considero que no deben estar expuestas directamente a internet por motivos de seguridad. Por último, resultará interesante el poder conectarme directamente a las máquinas desde mi computador a la red interna no expuesta a internet.

Teniendo en cuenta las características del entorno cloud que voy a crear, necesito integrar en la nube las siguientes redes:

- NAT: De esta red sólo forma parte la máquina NAT ya que es la red por defecto por la que `virtualbox[7]` provee a las máquinas virtuales de internet. Una vez que los paquetes llegan a través de internet a esta máquina son encaminados al nodo de computación y de control.
- Red Nat (*NatNetwork*): Esta red que nos entrega por defecto `virtualbox` se identifica con la red privada de la que hablábamos en la que se encuentran las tres máquinas (controller, compute y NAT). La máquina NAT recibe paquetes por la red externa y los envía a las máquinas a través de esta red privada.
- Red sólo anfitrión: Esta es una red (dada por defecto por `virtualbox`) que he creado yo por comodidad para conectar mi máquina física (portátil) con la máquina virtual que esté en ella. Esta red ha sido añadida a las tres máquinas de forma que pudiese conectarme a ambas sin necesidad de máquinas intermedias.

Las asignación de redes queda como se observa en las Figuras 3 y 4.

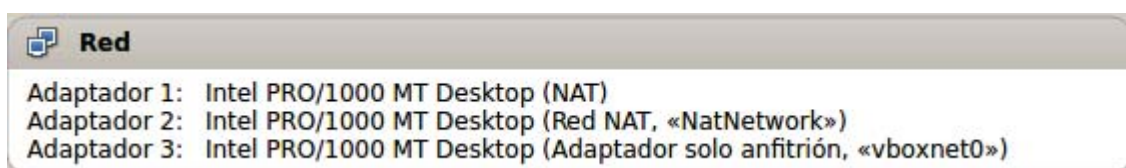


Figura 3. Redes presentes en la máquina NAT.

En la Figura 3 observamos cómo a través del Adaptador 1 llega internet a la máquina, mediante el Adaptador 2 se comunican todas las instancias entre sí y mediante el Adaptador 3 puedo conectarme desde mi ordenador a la instancia directamente (porque

se crea una red que incluye ambos). Estos Adaptadores actúan como tarjetas de red y en un entorno productivo se podría eliminar la tarjeta de red que aquí se muestra como Adaptador 3.



Figura 4. Redes presentes en las máquinas *Controller* y *Compute*.

En la Figura 4 se muestran los adaptadores necesarios para las máquinas *Controller* y *Compute*. En este caso ocurre lo mismo que en la *NAT*. La única diferencia es que se ven modificados los adaptadores porque a estas máquinas *virtualbox* no les provee de internet directamente ya que lo hace la máquina *NAT* y por ello se elimina el adaptador que provee a ésta de internet.

De este modo, el Adaptador 1 corresponde a la red en la que se comunican las máquinas virtuales entre si y el Adaptador 2 es la red que comunica mi ordenador físico con las máquinas virtuales como se observa en la Figura 5.

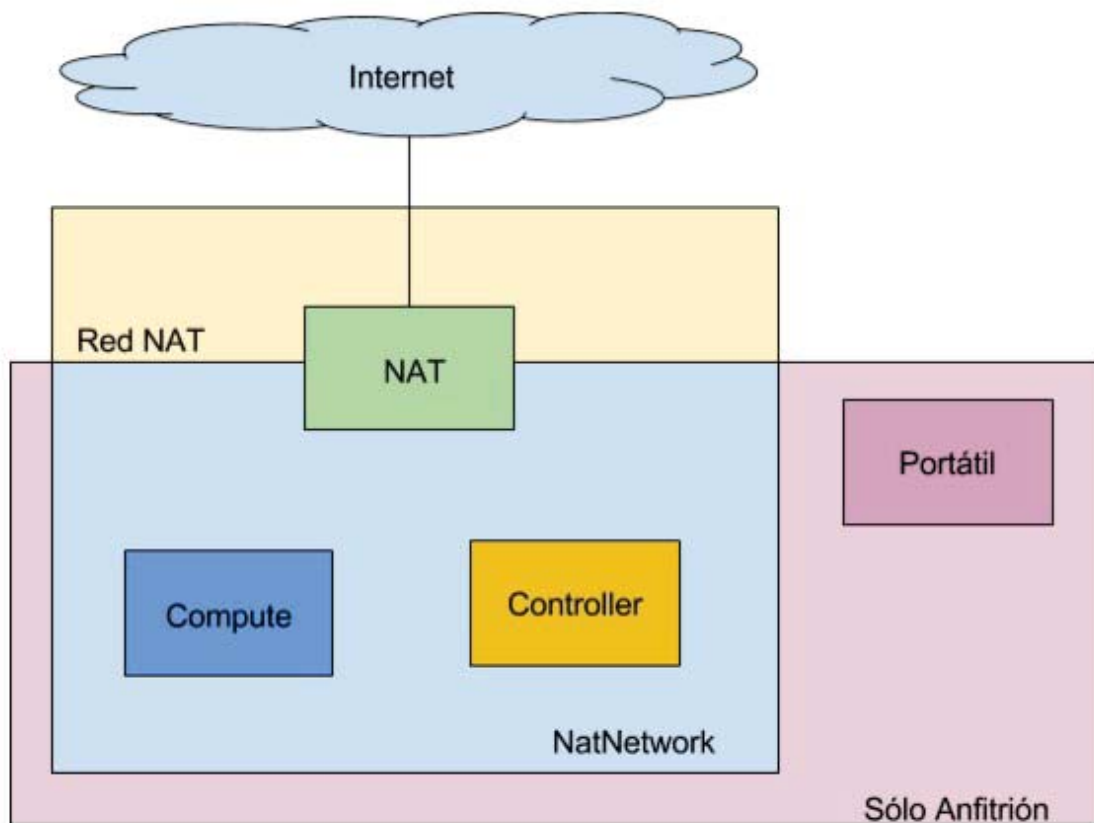


Figura 5. Representación de los adaptadores de red.

2.3- Hardware virtualizado

En este punto es necesario describir el hardware virtual que se emplea en la infraestructura así como sus requisitos para que soporte el sistema. Disponía de un hardware limitado como he mencionado anteriormente. Sobre todo, lo más preocupante era la memoria ya que la virtualización en si es uno de los recursos que más consume.

Para que el sistema funcionase correctamente debía asignar lo justo y necesario a cada máquina, y si luego me sobraban recursos, me servirían para mejorar el rendimiento. Por su parte, los discos duros virtuales, son de 32 GB cada uno.

Los recursos asignados a cada una de las máquinas virtuales son los siguientes:

NAT:

- Memoria RAM: 768 MB ya que esta máquina prácticamente tiene la función de enrutar paquetes y no consume en exceso memoria.
- Disco duro: 32 GB
- Memoria de vídeo: 12 MB ya que realmente tampoco necesitamos demasiado para llevar a cabo el proyecto. De hecho podríamos hacerlo todo en modo terminal.

Controller:

- Memoria RAM: 3072 MB ya que esta máquina es nuestro *core* en la instalación de *openstack*. Va a soportar todos los servicios y por ello va a consumir más recursos que el resto.
- Disco duro: 32 GB
- Memoria de vídeo: 12 MB ya que realmente tampoco necesitamos demasiado para llevar a cabo el proyecto. De hecho podríamos hacerlo todo en modo terminal.

Compute:

- Memoria RAM: 2048 MB ya que esta máquina es en la que se van a levantar los nuevos nodos de *openstack* que se creen. Como yo no voy a poder levantar múltiples por las limitaciones de hardware que tengo, con 2 GB lo veo suficiente. En un entorno productivo deberíamos añadir muchos mas recursos.
- Disco duro: 32 GB
- Memoria de vídeo: 12 MB ya que realmente tampoco necesitamos demasiado para llevar a cabo el proyecto. De hecho podríamos hacerlo todo en modo terminal.

2.4- Automatización

Llegados a este punto en el que tenemos clara la infraestructura a montar voy a introducir *ansible*. Esta es la herramienta que nos va a permitir la automatización de despliegue de los diferentes sistemas así como su configuración. Como es considerado gestor de configuración se pueden desplegar cambios en diferentes máquinas. Además es totalmente gratuito y tiene una versión gráfica y de pago por si fuese necesario en el caso de que el proyecto lo requiera. En cuanto a la estructura de archivos se refiere, *ansible* se basa en un fichero de inventario de *hosts*, unos ficheros ejecutables denominados *playbooks** y agrupaciones de tareas denominadas roles* [9] (hablando en un contexto muy básico).



Figura 6: Logo del orquestador Ansible

Para que los procedimientos de automatización que se lleven a cabo sean lo más óptimos posible en términos de eficiencia instauró unas políticas a tener en cuenta.

Las políticas que considero más importantes son:

- Código limpio
- Elegir el modo más corto posible de llevar a cabo una tarea
- Idempotencia en la ejecución

Una de las más importantes para mí es ésta última. Lo es porque la gracia de un gestor de configuración es poder realizar cambios en los archivos de configuración y desplegarlos para que se apliquen. Si los *playbooks* o los *roles* no son idempotentes en cada ejecución, no podrían estar haciéndose cambios una y otra vez porque la reproducción de las ejecuciones no sería determinista y habría errores al generar distintas configuraciones para una misma entrada.

*Playbook: Anexo 2

*Rol: Anexo 2

No se si ha sido por falta de tiempo o de conocimiento acerca de los elementos que necesito de `openstack`. Estos son los únicos elementos con los que he tenido más problemas para lograr la idempotencia (crear usuarios de los servicios de openstack, *endpoints* de API para realizar operaciones...), es decir, aquellas operaciones en las que intervienen los servicios de openstack. En el futuro me gustaría seguir estudiando la posibilidad de conseguir dicha idempotencia.

Para hacernos una idea de la estructura que toma el proyecto de `ansible`, es más o menos así:

`ansible.cfg` → Fichero de configuración de *Ansible*

`hosts` → Directorio en la que se almacenan los diferentes *hosts*

`dynamic_host_XXX` → Fichero de configuración de un *host*

`dynamic_host_YYY`

`dynamic_host_ZZZ`

`playbooks` → Directorio en la que se almacenan los *playbooks* a ejecutar

`vms.yaml` → *Playbook* que crea las máquinas virtuales

`openstack_NAT.yaml` → Se encarga de configurar e instalar la NAT

`openstack_CONTROLLER.yaml` → Se encarga de configurar e instalar el nodo **CONTROLLER**

`openstack_COMPUTE.yaml` → Se encarga de configurar e instalar el nodo **COMPUTE**

`encender_vms.yaml` → Se encarga de arrancar las máquinas virtuales.

`Roles` → Directorio en la que se encuentra cada uno de los roles

`vms` → rol encargado de la creación de las máquinas virtuales

`files` → Aquí se encuentran los ficheros necesarios del rol

`tasks` → En esta ubicación están las tareas a realizar

`templates` → Y aquí los templates para crear ficheros de configuración

CONTROLLER → rol del nodo controlador (misma estructura que *vms*)

COMPUTE → rol del nodo de computación (misma estructura que *vms*)

NAT → rol del nodo de NAT (misma estructura que *vms*)

Todos estos ficheros se encuentran escritos en formato *yaml* que es el que acepta por defecto *ansible*. En este lenguaje somos capaces de describir las tareas necesarias a realizar. Como se observa en la estructura anteriormente expuesta se han creado tres roles para cada uno de los nodos (o funcionalidades). El rol de *vms* lo que hace es levantar y configurar las máquinas virtuales necesarias en nuestra infraestructura. El del controlador lleva a cabo todas las tareas de instalación, configuración, etc. para que un *controller node* de *openstack* pueda estar corriendo y activo. Del mismo modo se procede con el *compute node*.

Todas las conexiones de *ansible* con las máquinas son realizadas a través de *ssh* (por el puerto 22) por lo que era importante permitir en el *firewall* de cada nodo el tráfico entrante por dicho puerto. Esto lo realicé en la imagen base que luego utilizo para levantar el resto de máquinas. Además fue necesaria la instalación del paquete *openssh-server* en nuestro *ubuntu 14.04.4*.

2.5- Control de versiones

Para llevar a cabo el control de versiones de los archivos así como tener el código en más de un lugar (a modo de *backup*) he decidido emplear un repositorio de *git* de los ofrecidos gratuitamente por *bitbucket*.

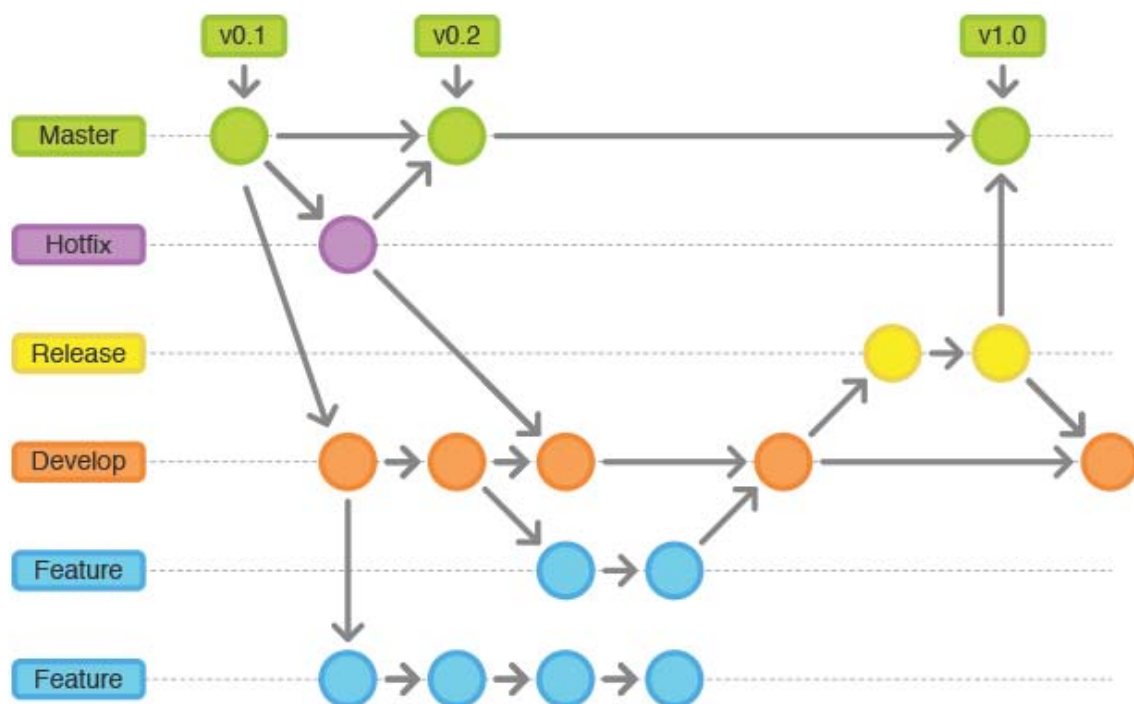


Figura 7. Gitflow. La rama máster se ha representado en color verde. El resto de nodos pertenecen a diferentes ramas sobre las que se han ido realizando sucesivos cambios.

Fuente: <https://www.atlassian.com>

Como vemos en la Figura 7 se ha seguido el *gitflow* recomendado. Se creó una rama *master* en la que están los cambios finales que han pasado las pruebas necesarias y otra rama de desarrollo en la que se iba desarrollando el nuevo código hasta llegar a versiones estables. Esto ha facilitado mucho las cosas en el sentido de ir afianzando el nuevo código que iba generando.

2.6- Openstack

Openstack es un proyecto de software libre y de código abierto que tiene como finalidad ofrecer un servicio de nube privada a los usuarios. Es una solución IAAS (*infrastructure as a service*) lo que quiere decir que se ofrece como servicio las máquinas, no un servicio, y no son gestionadas.



Figura 8. Logo representativo del proyecto openstack.

Para desarrollar el proyecto he elegido emplear la última versión estable de **openstack** para ubuntu, Liberty. Este proyecto realmente está compuesto de muchos subproyectos y todos ellos aglutinados constituyen un proyecto único que se ha denominado **openstack**.

Viendo el tiempo disponible para desarrollar este trabajo descarté la opción de integrar todos los elementos del proyecto porque sería inviable y no podría llegar a cumplir todos los objetivos que deberían plantearse en el despliegue de un sistema que entrara en producción en una situación real. Pero tampoco quería hacer exclusivamente lo más básico y que no se entendiese nada. Entonces decidí buscar un equilibrio entre tiempo disponible y el trabajo a desarrollar. Según la documentación los diferentes elementos de **openstack** se deben de instalar en un determinado orden porque en ocasiones tienen dependencias. En mi caso el equilibrio lo he basado entre la funcionalidad y hacer visible el proyecto. De este modo decidí que debía integrar todos los elementos hasta el *dashboard*. Este elemento gráfico creo que me permitirá mostrar los desarrollos realizados y que los lectores entendiesen mejor el proyecto. La funcionalidad tenía claro que debía basarse en el arranque o el lanzado de instancias, en la definición de redes (las cuales me propuse que hubiese privadas y públicas) y en el servicio de imágenes de las máquinas. Todo ello debía ser básico. Examinando la documentación vi que los elementos a instalar, configurar y automatizar serían los siguientes:

- **Keystone** (*identity service*): Es el servicio que proporciona la autenticación a los servicios mediante tokens.
- **Glance** (*image service*): Es el servicio que proporciona imágenes de las máquinas. Con ellas se debe poder levantar nuevas instancias, realizar backups, instalar máquinas en base a ellas...
- **Nova** (*compute service*): Es el servicio que permite la virtualización de las instancias de **openstack** en la máquina sobre la que se ejecuta este servicio. En nuestro caso vamos a crear una máquina de computación solo para que se levanten las instancias y se comunicará con el controlador mediante el servicio.

- **Neutron** (*networking service*): Es el servicio que aprovisiona las redes de las máquinas de **openstack**. Este quizá sea el servicio más complicado de configurar de todos los que tenga **openstack**.

- **Horizon** (*dashboard*): Es el servicio que permite mostrar toda la información de **openstack** mediante una herramienta gráfica. Además es muy útil ya que podemos realizar las operaciones necesarias a través de la interfaz web.

Para que quede todo esto más claro la Figura 9 muestra cómo se estructuran los servicios enumerados anteriormente.

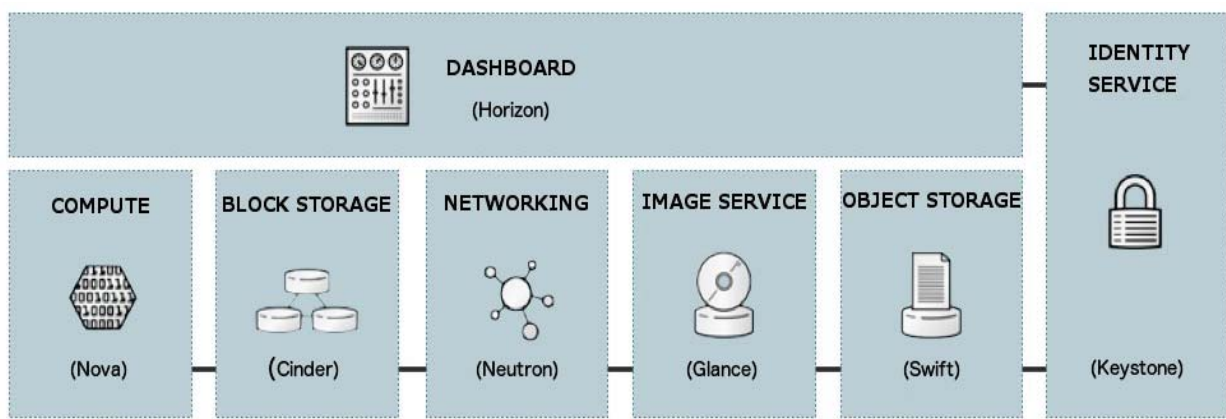


Figura 9. Capa de servicios de openstack

Fuente: http://docs.openstack.org/security-guide/_images/marketecture-diagram.png

Podemos comprobar que **horizon** es la capa gráfica que queda por encima de todos los servicios. Por otra parte **keystone** que es uno de los elementos importantes de **openstack** está posicionado como transversal a todos los servicios. Esto es así porque para cada petición que se hace a un servicio debe de autenticarse. Esta autenticación es manejada por **keystone** a través de tokens que genera a los peticionarios.

Para desplegar **openstack** existen unos requisitos previos que las máquinas deben cumplir. Estos se expondrán en las secciones de **Controller** y **Compute** del capítulo de implementación.

CAPÍTULO 3: IMPLEMENTACIÓN

3.1- NAT

Como he sugerido en páginas anteriores es uno de los elementos que podría ser prescindible, pero al ser un proyecto privado, creo que le aporta un nivel de seguridad adecuado. Este elemento ha sido el causante de fuertes quebraderos de cabeza para configurar las redes de `openstack`, pero el resultado es satisfactorio. Para comenzar a introducir el concepto partimos de la Figura 10.

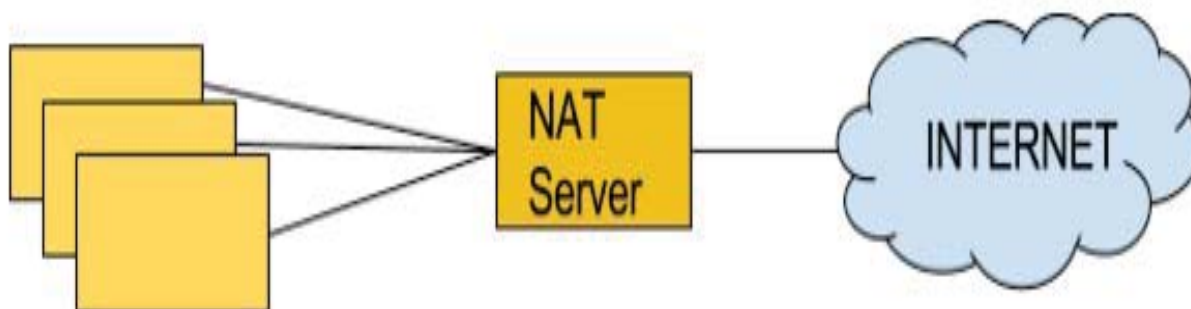


Figura 10. Configuración de red de un sistema con máquina NAT.

La NAT es una máquina que se encuentra entre internet y el resto de máquinas de nuestra infraestructura, de manera que tenemos dos redes diferentes. Existiría una red pública en la que la NAT recibe los paquetes de internet (mediante una IP pública) y los encamina hacia una red privada en la que se encuentran el nodo de control y el nodo de computación. Este encaminamiento de los paquetes entre la NAT y las máquinas se realiza a través de IPs internas o privadas (características de dicha red).

Además, esta máquina puede actuar como *firewall* de los paquetes y encaminarlos a cada una según unas reglas especificadas en el fichero de configuración `iptables`. Ahora pasemos a describir los elementos de configuración de la máquina.

Dentro de las configuraciones que he tenido que realizar en este proyecto ésta ha sido de las más sencillas porque solo era necesario su automatización al no haber elementos de

openstack que configurar.

Para llevarlo a cabo la solución más sencilla que se me ocurrió fue un *playbook* y un rol. En el *playbook* ejecuto las tareas necesarias en el servidor físico (en este caso mi portátil) y posteriormente llamo al rol.

Las tareas necesarias en el servidor son:

- Arrancar la NAT en modo *headless*, es decir, sin que se nos inicie gráficamente en el servidor.
- Instalar el paquete *sshpass* para poder acceder a las máquinas virtuales con *password* y modificar el archivo de configuración de *ssh* (*sshd_config*) para que no compruebe si ha cambiado el *host* (*host key checking*). De esta forma consigo que *ansible* no se quede parado en espera de la confirmación.
- Una vez que tenemos lo anterior aplico un tiempo de espera para asegurar que la máquina virtual se ha encendido correctamente y está funcionando. Esto lo hago para asegurarme de que no fallan los siguientes comandos que se ejecutan.
- Para poder ejecutar el rol de la NAT en la respectiva máquina virtual necesito generar dinámicamente el fichero de configuración del *host*. Para ello averiguo la ip de la máquina virtual gracias a la funcionalidad '*guestproperty get*' del comando *VboxManage* de *Virtualbox*[6]. Una vez tengo este dato puedo invocar un *template* previamente generado en el que se aplique el valor de esta variable. El *template* es como el de la Figura 11.

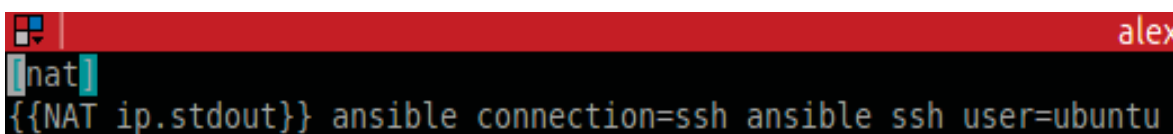


Figura 11. Template del host.

El valor de la IP se guarda en la variable *NAT_ip* que al invocar al *template* introduce su valor dejándonos un nuevo fichero de *hosts* para la máquina. Una vez tenemos esto *ansible* puede conectarse para ejecutar el rol. En el rol se especifican las tareas específicas que se deben ejecutar en una máquina para que cumplan ser de un determinado tipo (rol).

En este momento procedo a exponer las tareas que se cumplen en dicho rol para la máquina que se pretende configurar:

- Lo primero que hago es crear un *template* con la configuración del archivo de *interfaces* para asignarle una IP estática y de este modo conseguir una conexión segura al *host* en caso de que **ansible** tenga que hacerlo. En este fichero además restauro las reglas de *iptables* que veremos a continuación e introduzco un 1 en el archivo *ip_forward* (/proc/sys/net/ipv4/ip_forward) para que esta máquina pueda hacer *forwarding* de los paquetes que le llegan y redirigirlos.
- En cuanto a *iptables* básicamente hay dos reglas básicas. Una es hacer el *masquerade* de las ips. Es decir enmascarar las ip internas con la de la NAT. A efectos prácticos un paquete del nodo controlador o del nodo de computación parecería enviado por la propia NAT. La otra regla lo que permite es que la NAT envíe los paquetes que vienen del exterior al nodo correspondiente según el destino especificado en el paquete.

Específicamente las reglas son las siguientes:

- A POSTROUTING -o eth0 -j MASQUERADE
- A FORWARD -i eth1 -j ACCEPT

Partiendo de esto **ansible** aplica los *templates* y reinicia el servidor virtual quedando completamente configurado. Si se requiriese alguna configuración adicional se podría añadir sin problema en este rol.

3.2- Controller

Este apartado incluye la mayoría de la funcionalidad de **openstack**. Los elementos de **openstack** que contiene el *controller node* son los siguientes:

- Keystone
- Glance
- Nova

- Neutron

- Horizon

Para ello `ansible` lo he estructurado para que fuese lo más intuitivo posible. En primer lugar hay un *playbook* que se conecta a la máquina virtual y modifica la configuración de red para asignarle una IP fija. Posteriormente se conecta a la máquina mediante la nueva IP y ejecuta el rol en el que se ha programado las configuraciones y las instalaciones.

Como ocurría en la máquina de NAT se inicia la instancia `controller` en modo *headless*, se instala *sshpass* para la conexión con *password* y modificamos la variable *StrictHostKeyChecking* al valor 'no' para conseguir que no se compruebe el cambio de *host* y `ansible` pueda ejecutarse automáticamente sin problemas.

Una vez esperamos a que la máquina se ha levantado con el comando *vboxmanage* obtenemos la IP y la registramos en una variable de `ansible` para luego poder crear la plantilla de configuración (*template*) del *host* de forma dinámica. Ahora `ansible` ya puede conectarse a la máquina y proceder a modificar la IP.

Dicha modificación se realiza mediante diferentes *templates*. Se aplican tres *templates*, el del fichero de *hosts*, el archivo de interfaces con la nueva IP y el archivo de *hostname* sustituyendo en los ficheros de la máquina los valores correspondientes a nuestras configuraciones. Cuando la nueva configuración se introduce, se reinicia la máquina y se espera a que quede configurada con la nueva IP. Esto es posible gracias al módulo de `ansible` que queda recogido en la Figura 12.

```
- name: Wait for server to restart
  local_action:
    module: wait_for
    host=192.168.56.102
    port=22
    delay=1
    timeout=300
```

Figura 12. Módulo `wait_for` Ansible

Este módulo *wait_for* nos permite esperar hasta que arranque la máquina en la que estamos desplegando con `ansible`.

Llegados a este punto el *playbook* ejecuta el rol que pasamos a describir ahora.

3.2.1- Entorno

Para poder utilizar `openstack` con todos sus servicios asociados lo primero que debemos hacer es instalar el conjunto de dependencias generales que tiene. Estas dependencias son otros servicios básicos como pueden ser el de colas, sincronización horaria o bases de datos. Lo primero de lo que se encarga nuestro rol es de configurar el entorno de `openstack` según las directrices de la documentación oficial de *Liberty*. Se siguen para ello las siguientes instrucciones:

- Instalación de `chrony`. Este paquete consigue que la hora de todas las máquinas se sincronicen y no haya fallos por esas causas. En nuestro caso esta máquina va a ser el servidor NTP. Posteriormente el resto serán máquinas clientes sincronizadas con ésta como se observa en la Figura 13. Una vez configurada convenientemente se reinicia el servicio.

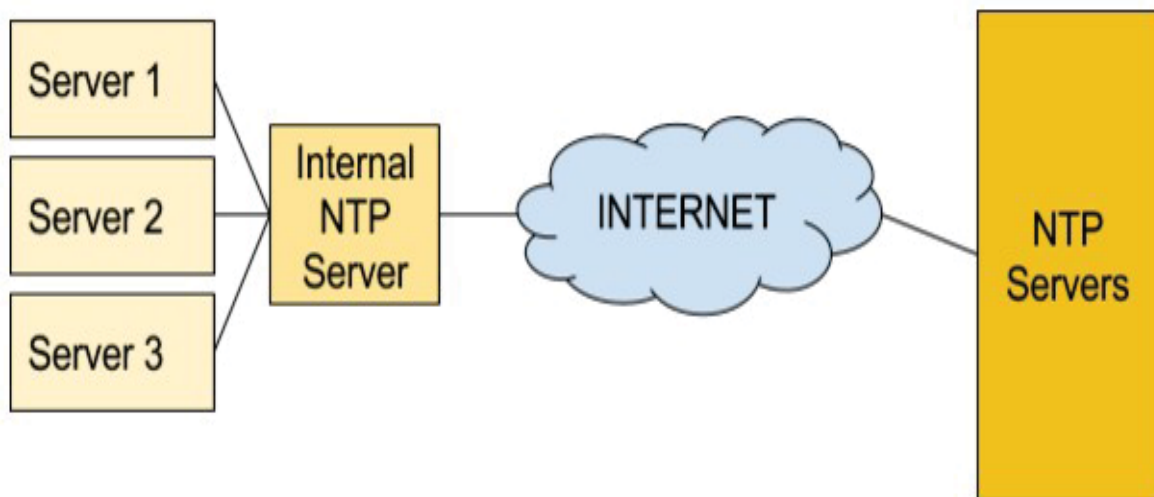


Figura 13. Sincronización NTP

- Instalación del paquete cliente de Openstack (`openstack-client`). `Ansible` actualizará los paquetes de las máquinas e instalará la última versión. Para hacer las pruebas más ligeras se actualizaron los paquetes en la máquina empleada como base y de esta manera se pudo omitir este paso.
- La mayoría de los servicios de Openstack emplean una base de datos para persistir los datos. Según la documentación podemos emplear `mysql` o

mongoDB. Nos decantamos por la última arbitrariamente. Ansible se encarga de instalar los paquetes y mediante un *template* realizar la configuración del mismo. Ponemos como IP en la que se levantará el servicio (*bind-address*) la IP de la máquina del *controller*. Para terminar reiniciamos el servicio.

- Openstack tiene un módulo de telemetría que aunque no ha sido insertado hasta el momento en mi proyecto, considero que la base debería estar para que solo haya que instalarlo y no tener problemas de dependencias. Por ello es necesario instalar una base de datos no relacional, en este caso mongoDB porque es la que se indica como idónea en la documentación.



Figura 14. Logo representativo de mongoDB.

Para ello se instalan los paquetes de mongoDB y se configuran mediante *templates* para poder realizar cambios si fuese necesario.

Complementando el entorno previo, Openstack emplea un sistema de colas para encolar los procesos. Con esta solución entiendo que buscan evitar puntos de fallo. El proyecto admite varias colas diferentes pero la documentación sugiere emplear rabbitMQ por compatibilidad con la mayoría de los sistemas.

Una vez se han instalado los paquetes procedo a crear un usuario para openstack y asignarle los permisos adecuados (de lectura y escritura).



Figura 15. Logo representativo de RabbitMQ.

3.2.2- Identity Service (Keystone)

El servicio de autenticación es un elemento centralizado de la infraestructura de openstack con el que interactúan el resto de elementos para realizar autenticación y autorización de servicios. Por ello cualquier servicio de openstack que recibe una petición de algún usuario o servicio comprueba si se tiene o no autorización para su uso. El flujo que sigue Keystone se representa en la Figura 16.

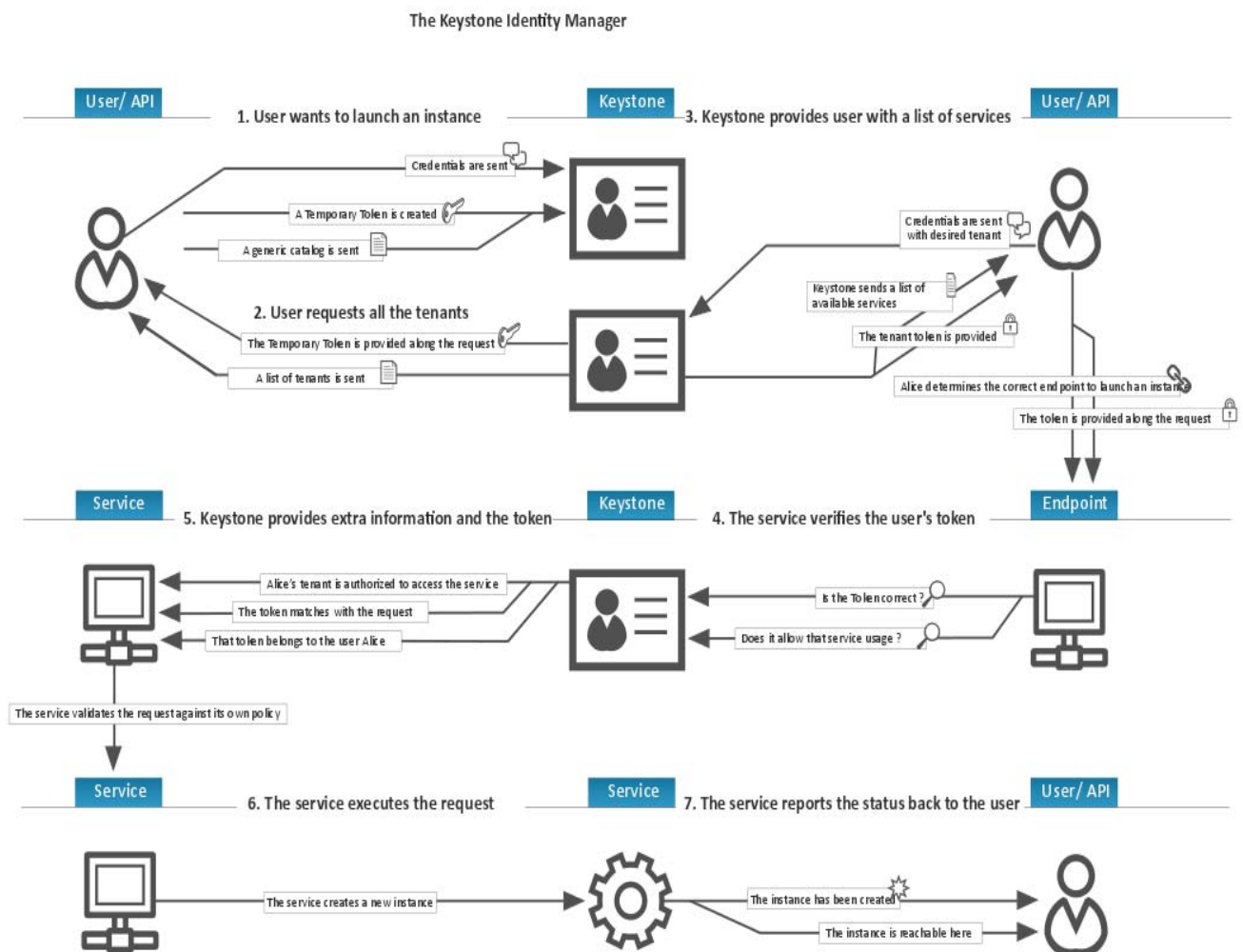


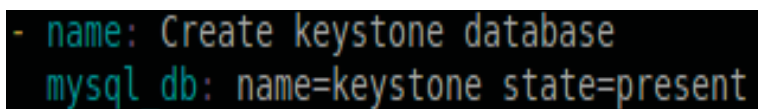
Figura 16. Flujo Keystone. Interacción de usuarios con API.

Fuente: <http://www.maquinasvirtuales.eu/componentes-de-openstack-openstack-identity-keystone/>

Keystone consta de tres partes diferenciadas que debemos conocer para entender su funcionamiento:

- Servidor: Es la base de el servicio que provee la autenticación y la autorización mediante una API REST.
- Drivers: Son elementos donde se guardan las credenciales para que el servicio pueda comprobarlas (base de datos).
- Módulos: Cada uno de los componentes que utilizan el servicio. Interceptan las peticiones, extraen las credenciales y las envían al servidor central para que se compruebe si son correctas.

Para llevar a cabo la instalación y configuración del servicio lo que primero hay que hacer es crear su base de datos. Como escribí en líneas anteriores instalamos `mariaDB` para poder hacer estas operaciones. Como `ansible` tiene un módulo para `mysql` el proceso de creación de base de datos es muy sencillo así como la asignación de privilegios sobre el usuario que hay que crear. Para ilustrar esto la Figura 17 muestra la facilidad para crear una base de datos.



```
- name: Create keystone database
  mysql_db: name=keystone state=present
```

Figura 17. Creación de base de datos con Ansible

Una vez hecho esto en la documentación se indica que hay que eliminar el arranque automático del servicio. Esto lo hacemos mediante un *template* del archivo `keystone.override`.

Ahora ya podemos pasar a instalar los paquetes que necesita `keystone`, los cuales son los siguientes:

- `keystone`
- `apache2`
- `libapache2-mod-wsgi`
- `memcached`
- `python-memcache`

Una vez se han instalado pasamos a hacer la configuración del servicio. Para simplificarlo lo hacemos con *templates* por si existen cambios poder llevarlos a cabo sin

problemas. En estos *templates* tenemos que añadir un token al que se le denomina 'ADMIN'. Añadimos la configuración de la base de datos para que el servicio se pueda conectar y configuramos donde se encontrará levantado el servicio de *memcached* para que pueda comunicarse.

Para conseguir que la resolución de futuros problemas (en caso de haberlos) sea más sencilla, activo la opción *verbose* en el *template* para que muestre más información de *debug*.

Por último pasamos el *virtualhost* de *apache sites-enabled* y reiniciamos el servidor para asegurar el correcto funcionamiento de todos los servicios. La configuración de *apache*, por su parte, nos viene dada en la documentación oficial del servicio.

Cuando ya está corriendo en la máquina de manera correcta procedemos a crear los servicios y los *endpoint* de las API. Aquí se me presenta un problema y es que según indica la documentación oficial, la *url* del *controller*, la versión de la API y al *admin token* se deben exportar como variables de entorno. Para ello según he podido comprobar de varias fuentes se crea un *script* que haga el *export* de las variables de entorno con dichos datos. En mi caso tenía que encontrar una solución alternativa ya que con *ansible* quedaría bastante “feo” ejecutar eso como un *script* porque no es la finalidad primaria de este software. Por ello, navegando en la documentación de *ansible* encontré un módulo de configuración del entorno del sistema, denominado *environment*, con el cual se pueden añadir variables de entorno. Esto se realiza del siguiente modo en codificación la *yaml* de *ansible*:

```
- name: 'Aquí se introduce el nombre de la tarea'  
  task: 'Cambiando el tipo de operación que se requiera'  
  environment:
```

```
    OS_TOKEN: 'XXXXXXXXXXXX'  
    OS_URL: 'http://controller:35357/v3'  
    OS_IDENTITY_API_VERSION: '3'
```

```
  tags:  
    - keystone
```

Como podemos comprobar en el fragmento anterior, en el apartado *environment* de esa tarea introducimos unas variables. Éstas serán tomadas como si fuesen variables de entorno sin la necesidad como comentaba anteriormente de ejecutar ningún *script* que las exporte.

Otra de las funcionalidades que ofrece *keystone* es llevar a cabo el catálogo de

servicios que tiene nuestra infraestructura de `openstack` y por ello debemos proceder a dar de alta el propio servicio de autenticación.

Cuando el servicio ya se ha dado de alta en `keystone` podemos empezar a crear los *endpoint* de las API de este. `Openstack` siempre necesita 3 *endpoints* diferentes para un mismo servicio. Son los siguientes:

- *Admin*: Este *endpoint* permite modificar y gestionar usuario y permisos mediante la API. Sólo debería ser manejado por los administradores de la infraestructura de `openstack`.
- *Public*: Con ella se pueden hacer las mismas operaciones que con la Admin menos el manejo de usuarios y permisos. Esta estaría abierta a internet para que los usuarios pudiesen llevar a cabo operaciones.
- *Internal*: Esta está limitada a los *host* que contengan servicios de `openstack` instalados. A través de ella pueden comunicarse, autenticar, ver los catálogos de servicios, etc.

Al igual que ocurrió para dar de alta el servicio necesitamos utilizar el módulo de *environment* de `ansible` para pasar las variables de entorno necesarias para establecer la conexión. Además hay que destacar que las llamadas para la creación de los *endpoints* son a la misma url pero en diferente puerto. Esto es porque como acabo de explicar, los *endpoints* de administración y públicos son diferentes.

Ahora bien para utilizar todo esto que hemos creado `openstack` emplea usuarios, proyectos, roles, y dominios. Como dominio vamos a emplear el que está por defecto en `openstack`. Lo ideal sería disgregar los permisos de usuarios y roles, pero para ver que todo funciona correctamente, decido hacerlo como superusuario. Para ello tengo que crear el usuario 'admin', el proyecto 'admin', el rol 'admin', y por último asocio el rol admin con el proyecto. Esto lo he marcado con un *tag* de `ansible`. Este *tag* nos permite ejecutar en el *playbook* o rol sólo las líneas que están marcadas con dicha etiqueta. Este es un modo muy sencillo de realizar pruebas ya que no tenemos que esperar a que se ejecuten todas las tareas, sino que sólo lo harán las que están marcadas.

Según la documentación oficial para comprobar que funciona convenientemente el servicio de `keystone` podemos hacerlo con el siguiente mandato:

```
openstack --os-auth-url http://controller:35357/v3 \
--os-project-domain-id default --os-user-domain-id default \
--os-project-name admin --os-username admin --os-auth-type password \
token issue
```

Como observamos, se conectaría al servicio pasando el dominio *default*, en usuario debemos poner el que hemos creado (admin), así como el proyecto admin que creamos. Esto nos debería contestar con un token y mostrarnos su información.

3.2.3- Image Service (Glance)

El servicio de imágenes de `openstack` nos permite almacenar y utilizar imágenes de máquinas virtualizables. Con ellas podremos levantar máquinas de `openstack` en base a esa imagen previa. Además este servicio puede darnos otros adicionales como el de *backups*. Al poder realizar imágenes de máquinas podríamos emplearlas como respaldo. Imaginemos que tenemos una máquina con todos los servicios necesarios instalados y configurados. Podremos realizar una imagen y guardarla de forma que si existe algún fallo podríamos restaurarla en base a ella. Puede ser un método de recuperación ante desastres.

Para el almacenamiento de dichas imágenes en esta práctica lo vamos a hacer en la propia máquina pero como un posible evolutivo estaría bien introducir el servicio de almacenamiento de `openstack` en otro nodo. Así podríamos tenerlas en el de un modo más correcto.

Este servicio consta de cuatro elementos para su funcionamiento que son los siguientes:

- **Glance API**: Sirve para guardar o recuperar las imágenes.
- **Glance registry**: Sirve para guardar o recuperar los metadatos asociados a las imágenes.
- **Base de datos**: Sirve para guardar los metadatos asociados a las imágenes.
- **Repositorios de datos**: Es el lugar físico donde vamos a guardar nuestras imágenes. En primera instancia utilizaremos el servidor local pero también podría ser un evolutivo, como comentaba, modificarlo en el futuro.

Una vez que tenemos claros estos conceptos podemos proceder a instalar y configurar los componentes. Al igual que en el servicio anterior necesitamos crear una base de datos y un usuario para que el servicio pueda persistir los datos. Para ello, con el módulo de `ansible`, lo creamos y damos permisos al usuario sobre la base de datos.

El siguiente paso que tenemos que realizar es crear el usuario de `openstack` del servicio así como asignarle el rol que le queramos asignar. Según la planificación debemos darle el rol admin ya que vamos hacerlo todo con ese rol. Con la ayuda del

modulo de entorno de `ansible` podemos llevarlo a cabo. Las variables que debemos tener en cuenta son las siguientes:

- OS_PROJECT_DOMAIN_ID: default
- OS_USER_DOMAIN_ID: default
- OS_PROJECT_NAME: admin
- OS_TENANT_NAME: admin
- OS_USERNAME: admin
- OS_PASSWORD: 'XX'
- OS_AUTH_URL: http://controller:35357/v3
- OS_IDENTITY_API_VERSION: 3

Como en todos los servicios necesitamos darlo de alta en `openstack` y crear los *endpoint* de las API que se van a emplear. Si todo ello se desarrolla satisfactoriamente podremos pasar a instalar los siguientes paquetes:

- glance
- python-glanceclient

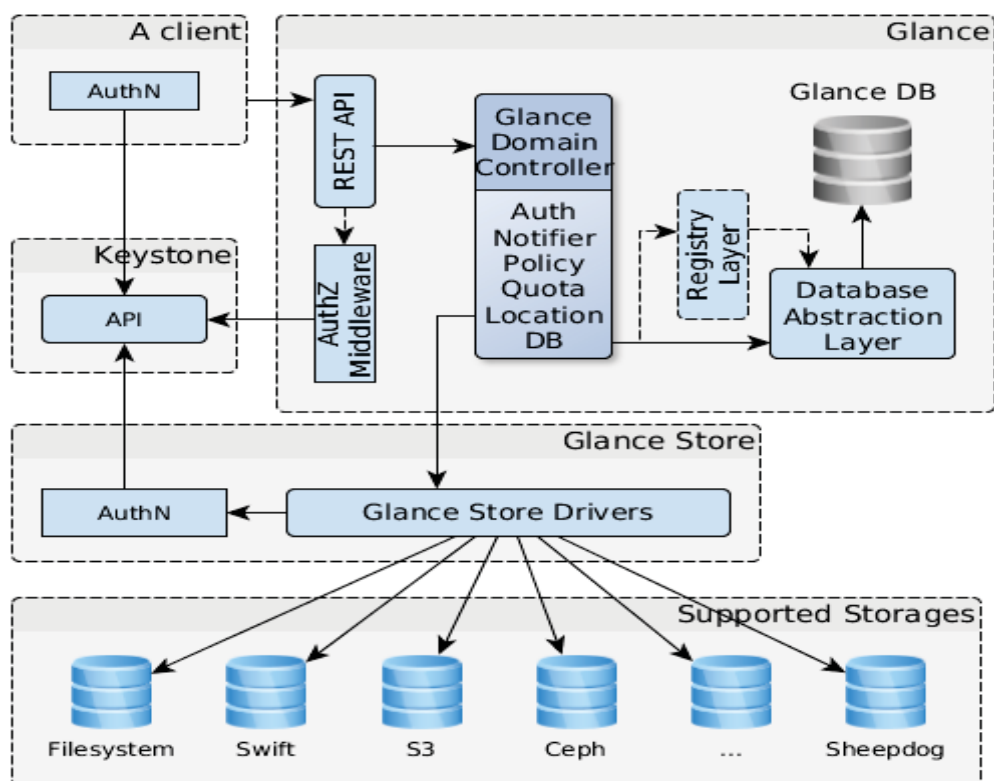


Figura 18. Glance flow. Petición mediante la API empleando keystone para autenticar.

Fuente: http://docs.openstack.org/developer/glance/_images/architecture.png

Para configurarlo todo tenemos que modificar los archivos `glance-api.conf` y `glance-registry.conf`. Esto lo realizo mediante *templates* para poder cambiar la configuración en un momento dado. Para tener más claro el modo de configurarlo veamos como trabaja `glance` (Figura 18). Ahora sí pasamos a configurar los archivos:

`Glance-api.conf:`

- En primer lugar en la sección *database* debemos incluir los datos de conexión del usuario que hemos creado a la base de datos creada. Sin ello el servicio no se podría conectar.
 - En la sección `keystone_authtoken` debemos tener las url de *keystone* con el método de verificación de la autenticación (*password* en este caso) e introducir los datos del dominio, proyecto, usuario y *password*.
- Además debemos especificar el tipo de autenticación que vamos a emplear (*keystone*).
- Para la parte de almacenamiento (*glance-store*) debemos poner como `default_storage` 'file'. De este modo conseguimos que el almacenamiento de las imágenes sea en local.
 - Por último ponemos la opción de *verbose* a 'true' para que nos invoque la máxima información posible.

`Glance-registry.conf:`

Más o menos la configuración es similar al archivo anterior pero para `api-registry`.

- Datos de conexión a base de datos
- Configuración de *keystone*
- Configuración del *verbose* para que nos muestre la mayor información posible en caso de error

En último lugar hacemos público el servicio de base de datos de `glance` para que se comience a usar y reiniciamos los servicios.

3.2.4- Compute Service (Nova)

Es el servicio de computación de `openstack` permite ofrecer la infraestructura como IAAS. Es el elemento principal de todos estos módulos que estoy exponiendo ya que es el encargado de controlar los *host* virtuales o lo que se denomina “la nube”. Como ya comentaba en la introducción, y aquí vamos a poder observar, la nube no deja de ser un conjunto de servidores físicos con software de virtualización. En este caso `openstack`.

Este servicio de computación interactúa con el resto de módulos de `openstack` para conseguir un fin común. El servicio de autenticación se encarga de darle los tokens para permitirle posteriormente llevar a cabo acciones, el de imágenes le permite levantar máquinas en base a ellas, el *dashboard* (del que hablaremos posteriormente) le permite dar gráficamente los recursos que se están empleando, etc.

Un elemento importante que he visto en la documentación y creo que es reseñable es la capacidad de escalamiento horizontal. Tradicionalmente (y me consta que aún hoy en determinados sitios se hace) el modo de escalar era vertical. Esto quiere decir que si por alguna razón era necesario el escalado de la infraestructura se introducían mas recursos a las maquinas para que tuviesen más rendimiento. ¿Cuál es la filosofía de `openstack` en este sentido? Todo lo contrario. `Openstack` apuesta por el escalado horizontal sin necesidad de recursos innecesarios. En lugar de tener una máquina con muchos recursos que se traduce en algo muy caro se puede apostar por varias máquinas más pequeñas trabajando en paralelo, lo cual es mucho más barato. ¿Qué conseguimos con esto? Pues fácil, rendimiento a un costo mucho menor.

Este servicio ya consta de bastantes más elementos que los anteriores. Son los que paso a describir a continuación:

- `Nova-api service`: Se encarga de las peticiones de las API de computación. Además de aceptar la API de `openstack` un elemento curioso es que acepta la API de Amazon EC2.
- `Nova-api-metadata service`: Esta API se encarga de responder a las instancias de `openstack` con metadatos de las mismas.
- `Nova-compute service`: Es un demonio que se encarga de crear y eliminar las instancias de `openstack` a través de la API del `hypervisor`. En mi caso como voy a configurar `openstack` como si mi portatil (servidor) no soportara aceleración hardware emplearé `libvirt` con `QEMU` en lugar de con `KVM`. La opción de `QEMU` viene dada por recomendación de la documentación.

- `Nova-scheduler service`: Sirve para determinar en que servidor de computación se encuentra una instancia.
- `Nova-conductor module`: Es un intermediario entre el servicio de nova y la base de datos con la finalidad de que el propio servicio no tenga acceso.
- `Nova-cert module`: Es un servicio empleado para crear certificados necesario sólo para la API de Amazon.
- `Nova-network worker daemon`: Similar al servicio `nova-compute service`
- `Nova-consoleauth daemon`
- `Nova-novncproxy daemon`: Ofrece un proxy para acceder a las instancias mediante VNC
- `Nova-spicehtml5proxy daemon`: Ofrece otro proxy para acceder a las instancias a través de conexiones SPICE.
- `Nova-xvpvncproxy daemon`: Igual que `Nova-novncproxy daemon`
- `Nova-cert daemon`: Servicio de certificados x509
- `Euca2ools client`: Interprete de línea de comandos para controlar los recursos del cloud.
- `Nova client`: Permite a los usuarios emplear comandos
- `Queue` (cola): Implementado con `RabbitMQ` aunque se puede implementar con otro tipo de colas.
- Base de datos `SQL`: Para persistir los tiempos de ejecución y de vida de los recursos del *cloud*.

Teniendo en cuenta todos estos elementos, ¿cómo se hace una operación global en `openstack`? Es una operación compleja en la que interactúan muchos elementos diferentes. Se puede comprobar en la Figura 19 como se aprovisiona una instancia.

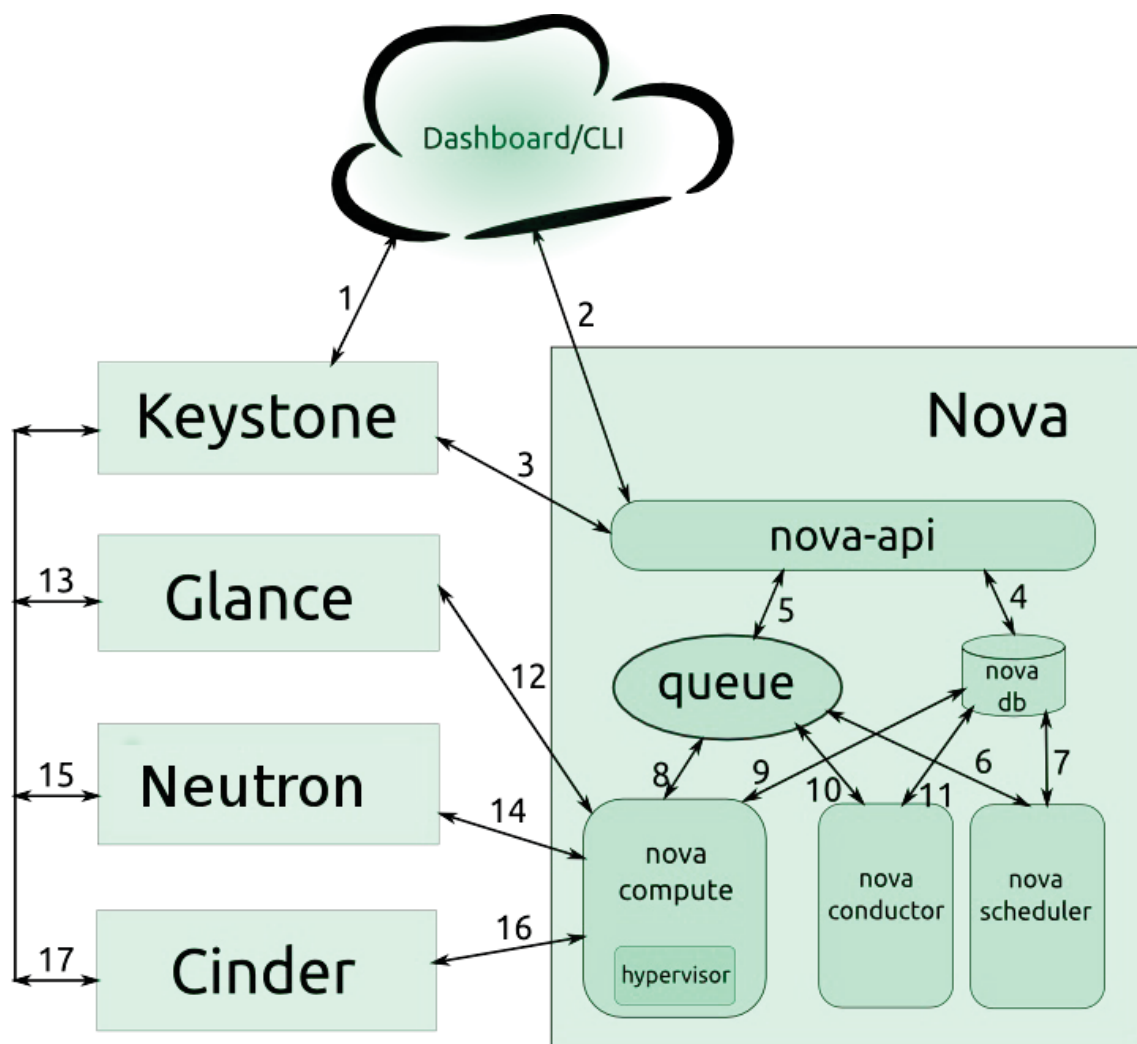


Figura 19: Aprovisionamiento de una instancia con autenticación mediante keystone, imagen provisionada por glance y redes por neutron.

Fuente: <http://docs.openstack.org/icehouse/training-guides/content/figures/11/a/figures/image02.png>

Teniendo claros todos los elementos podemos comenzar con la instalación y configuración de los componentes. En primer lugar debemos crear una base de datos para el servicio así como un usuario.

Lo siguiente a tener en cuenta es la creación del usuario de `openstack` para el servicio (`nova`). Además como ya escribí algunas líneas más atrás el rol que debemos dar a este usuario será el de `admin`. Si tratamos de hacer esto desde `ansible` con el comando normal de `openstack` dará error porque faltarán las variables de entorno que se configuran según la documentación con el script que ofrecen. Yo personalmente emplearé el módulo de entorno de `ansible` que ya he mencionado en puntos

anteriores.

Ahora ya podríamos crear los *endpoints* de las API del servicio que como en anteriores ocasiones son las tres siguientes: Public, Internal, Admin.

Teniendo esto creado pasamos a la instalación de paquetes para lo cual necesitamos hacer uso del módulo `apt` de `ansible`. Con él lo que conseguimos es que nos instale la versión oficial del repositorio sin necesidad de descargarlo por nosotros mismos. Esto nos vale porque la máquina es un `ubuntu` pero si lo pensásemos para otra distribución como `centOS` o `rhel` no nos valdría. En estos casos, en vez del módulo `apt` deberíamos usar el `yum`. Navegando por la documentación de `ansible` posteriormente encontré que hay un módulo que podría valer si fuesen concretamente los mismos paquetes a instalar en ambos. Ese módulo se denomina *package*.

Siguiendo el hilo de la instalación de los paquetes los necesarios para que el servicio funcione son: `nova-api`, `nova-cert`, `nova-conductor`, `nova-consoleauth`, `nova-novncproxy`, `nova-scheduler` y `python-novaclient`.

Transcurrido el tiempo de instalación hay que realizar las configuraciones pertinentes en el archivo de configuración de nova (`nova.conf`). Como en el resto de servicios lo primero que debemos configurar es la conexión a la base de datos con las credenciales creadas para que el servicio se pueda comunicar con ella.

Después hay que modificar la configuración de la cola para que funcione con `rabbitMQ` en la sección *default* y en la sección `oslo_messaging_rabbit`. Para la autenticación hay que configurar `keystone` de manera que se puedan conectar mediante usuario y contraseña.

Añadimos la IP de la máquina servidora en la configuración `DEFAULT` y además en la sección de `VNC` para que trabaje correctamente. Configuramos el *host* de `glance` (servicio de imágenes) y añadimos la opción *verbose* para sacar el mayor número de errores posibles en caso de que los hubiese. Teniendo todo esto, inicializamos la base de datos del servicio y reiniciamos todos los servicios de nova que hemos instalado.

Según la documentación hay unos pasos a seguir para ver que el servicio funciona. Como de momento sólo hemos instalado la parte del `controller` no nos va a salir de forma correcta ya que aún falta la parte del `compute`. Pero el resto de componentes debería dar '*enabled*' como resultado.

Para comprobar esto basta con hacer un '`nova service-list`' y nos lo muestra. Por otra parte con el comando '`nova endpoints`' podemos ver los *endpoint* de las API que han sido creadas.

3.2.5- Networking Service (Neutron)

Este servicio es el encargado de proveer redes a las diferentes instancias y servicios que se creen en `openstack`. Pueden ser de varios tipos y el servicio en si está compuesto por los siguientes elementos:

- **Neutron-server**: Es el encargado de estar escuchando en la API para mandar cada petición que se le realice al *plugin* correcto.
- **Plugins**: Crean redes, subredes, asignan ip, puertos...
- Cola de mensajes: Sirve para enrutar la información entre el servidor de `neutron` y el resto de agentes.
- Base de datos: Sirve para almacenar el estado de la red en algunos *plugins* particulares.

Los elementos virtualizados que nos puede ofrecer `neutron`, así como sus interacciones, son redes, subredes y routers.

Lo primero que debemos hacer para empezar a configurar el servicio de red es crear la base de datos del servicio. Lo realizaremos como lo venimos haciendo hasta ahora con el módulo de `ansible`. Además de la base de datos debemos crear un usuario para que gestione ésta.

Ahora ya podemos crear el usuario de `openstack` del servicio y asignarle un rol. El usuario lo llamaremos del mismo modo que al servicio y le aplicaremos el *rol* admin. Con ello creado ya podemos empezar a trabajar con la API. Daremos de alta el servicio en `openstack` y crearemos los *endpoint* de las API. Como en el resto de servicios debemos crear tres *endpoints* diferentes: public, internal y admin.

En este momento debemos tomar una determinación sobre el tipo de red que montaremos. Con `openstack` se abren dos opciones para ello:

- **Provider networks**: Son las redes más sencillas en las cuales no se pueden crear redes privadas, IP flotantes, etc. Emplean la misma red que el servidor sobre el que se crea.
- **Self-service networks**: En este caso si que se permiten crear redes privadas para las instancias que se lancen. Además nos permite conectar estas con la red pública mediante routers y añadir IP flotantes a las instancias para que puedan ser referenciadas desde internet.

En primera instancia decidí montar la opción de *'providers'* porque era más sencillo y quería ver la funcionalidad pero creo que lo acertado es emplear el segundo tipo ya que nos puede aportar una funcionalidad real. Por ello, cuando entendí bien el sistema implementé el segundo tipo.

Lo primero que debemos hacer es instalar los paquetes necesarios para que el servicio funcione: `neutron-server`, `neutron-plugin-ml2`, `neutron-plugin-linuxbridge-agent`, `neutron-l3-agent`, `neutron-dhcp-agent`, `neutron-metadata-agent`, `python-neutronclient` y `conntrack`.

Y después procedemos a configurar cada uno de los paquetes instalados mediante *templates* con el formato demandado por ansible. Comenzamos configurando `neutron` con su archivo de configuración (`neutron.conf`). En él realizamos los siguientes cambios y configuraciones:

- En primer lugar debemos introducir los datos de conexión a la base de datos con el usuario que hemos creado.
- Habilitamos el *plugin ml2* y configuramos las conexiones con las colas de `rabbitMQ`.
- Configuramos el acceso contra `keystone` para que se pueda producir la autenticación. Introducimos el *endpoint* del servicio así como usuario, contraseña, proyecto y dominio.
- El siguiente paso es la parte de `nova` (computación). Al igual que con `keystone` añadimos *endpoint* y credenciales. Añadimos la opción *verbose* para obtener más información.

Teniendo completada esta configuración general es necesario configurar los *plugins* para que funcionen bien. Son los siguientes:

- **Modular Layer 2 (ML2)**: Se encarga de crear las redes virtualizadas para las instancias. Para ello empleo (guiado por la documentación) redes `VXLAN` que son soportadas con el mecanismo de `linuxbridge`. Habilitamos el driver de seguridad de puertos y habilitamos `ipset` (para incrementar la eficacia de las reglas de los grupos de seguridad).
- **Linux bridge agent**: Crea infraestructuras de red virtuales para las instancias como pueden ser los túneles `vlan` para las redes privadas. Las modificaciones debemos hacerlas en su archivo de configuración: `linuxbridge_agent.ini`. En él añadimos la interfaz que actúa como

pública y la IP interna de la máquina como IP local. Por último habilitamos los grupos de seguridad y configuramos el driver del *firewall*.

- `Layer 3 agent`: Se configura la interfaz del driver
- `DHCP agent`

Esto es todo en cuanto a la configuración del propio servicio de `neutron` pero debemos configurar los servicios que interactúan con él para que funcionen correctamente los otros servicios. En este caso debemos configurar `nova` (ya que es la parte de computación), que es dependiente de `neutron`. Para ello en el archivo de configuración de `nova` (`nova.conf`) debemos añadir las url donde se encuentra `neutron`, usuario, *password*, región...

Por último publicamos la base de datos del servicio siguiendo las instrucciones que nos da la documentación y reiniciamos los servicios.

3.2.6- Dashboard Service (Horizon)

`Horizon` nace de la necesidad de acercar al usuario final la funcionalidad de `openstack` de un modo gráfico y sencillo. No tendría sentido que un operador tuviese que estudiarse toda la infraestructura de `openstack` sólo para ser un usuario del sistema. Para todos los temas relacionados con ella debería de haber un grupo de administradores de sistemas especializados en la materia.

Para suplir la necesidad de la que tratamos, lo más interactivo es el uso de una página web que desencadene eventos que interactúen con `openstack`. De eso trata este módulo.

En esta infraestructura que no va a ser un entorno productivo puede estar bien montarlo con `Apache` pero como un evolutivo creo que sería interesante configurarlo con `nginx` y ver si mejora el rendimiento.

La interfaz gráfica que nos proporciona este módulo es la correspondiente a la Figura 20 y 21.

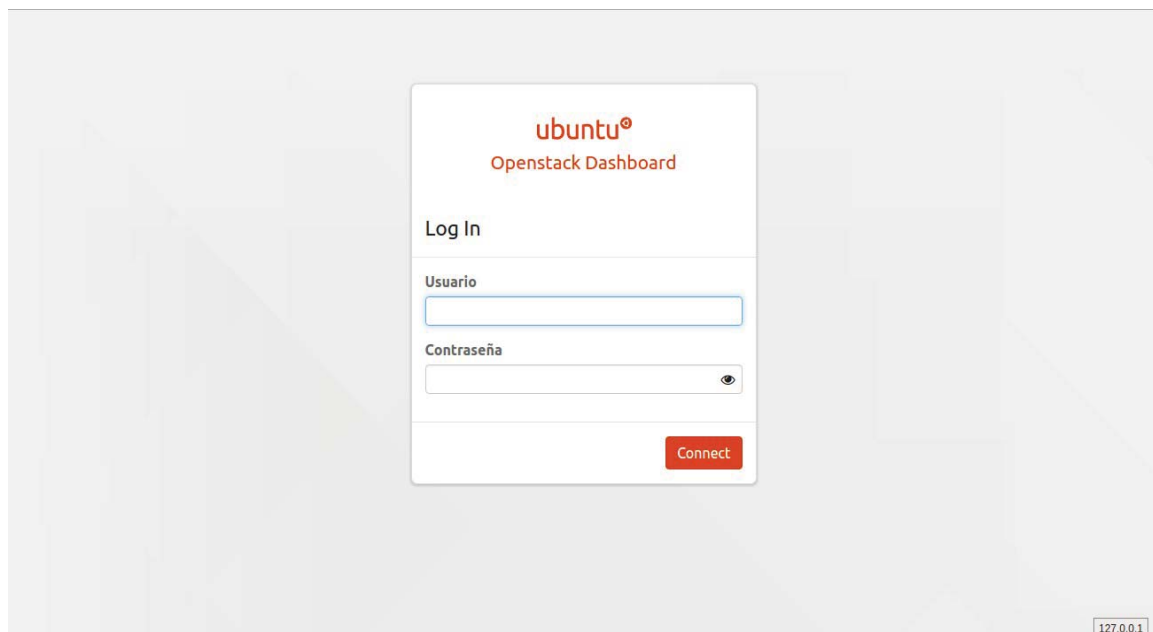


Figura 20. Dashboard login.

Vista general

Usage Summary

Select a period of time to query its usage:

From: 2016-06-01 To: 2016-06-01 [Submit](#) The date should be in YYYY-mm-dd format.

Instancias Activas: 1 Active RAM: 512MB This Period's VCPU-Hours: 21,04 This Period's GB-Hours: 21,04 This Period's RAM-Hours: 10774,72

Uso [Descargar resumen en CSV](#)

Nombre del proyecto	VCPU	Disco	RAM	Horas VCPU ?	Horas disco GB ?	Horas Memoria MB ?
admin	1	1GB	512MB	21,04	21,04	10774,72

Displaying 1 item

Figura 21: Vista general del proyecto.

Como se puede comprobar en la imagen sigue la línea de ubuntu en cuanto a estilos pero según las pruebas realizadas aún muestra algunos fallos con las traducciones. Por lo general cumple la finalidad y las tareas que se hacen desde el *dashboard* funcionan satisfactoriamente.

Horizon interactúa con todos los servicios instalados porque cumple las funciones de cada uno de ellos. El paquete a instalar se encuentra en los repositorios y se denomina *openstack-dashboard*. En cuanto a las configuraciones que hay que llevar a cabo son las siguientes:

- En el archivo `local_settings.py` debemos introducir cual va a ser el *host* que va a soportar el módulo (en este caso el *controller*) y dar permisos a los usuarios que se desee. En nuestro caso vamos a permitir a todos.
- Configurar *memcached* según los parámetros que nos indica la documentación:

```
CACHES = {
    'default': {
        'BACKEND': 'django.core.cache.backends.memcached.M
                    emcachedCache',
        'LOCATION': '127.0.0.1:11211',
    }
}
```

- Introducir roles y versiones de las API
- Recargar la configuración de apache

En este momento se debería poder acceder mediante la url <http://controller/horizon>. Desde la máquina *controller*. Si accedemos desde otro lugar debemos cambiar la url sustituyendo *controller* por la IP de la máquina.

3.3- Compute

El nodo de computación es aquel en el que posteriormente se van a desplegar las instancias virtuales de *openstack*. En este caso he decidido poner un único nodo

porque como el servidor será mi portátil sería absurdo añadir más por requisitos. Pero como indico podrían ser varias máquinas.

En definitiva este nodo se va a encargar de recepcionar peticiones desde el `controller` y llevarlas a cabo. Para toda la instalación y configuración son necesarios los siguientes pasos.

3.3.1- Entorno

Al igual que ocurría con el nodo controlador en este nodo es necesario preparar el entorno para poder afrontar la instalación de los servicios que requiere `openstack`. En este caso la documentación nos sugiere que es necesario añadir un disco de mayor tamaño que en el nodo configurado anteriormente.

Teniendo en cuenta estos detalles podemos comenzar a configurar el entorno con lo necesario. Como hicimos con el nodo controlador necesitamos un mecanismo de sincronización horaria y para ello empleamos NTP. En este caso el servidor es el `controller` como indicamos previamente en esta documentación. Por ello debemos configurar la máquina de computación como un cliente. Si tuviésemos más nodos todos deberían ser clientes del controlador. Dicho esto una de las tareas de `ansible` será instalar `chrony` así como configurarlo como cliente mediante *templates*. La configuración para el cliente debe ser como sigue:

- `chrony.conf`: server controller iburst

Con ello reiniciamos el servicio y queda listo.

Para habilitar la descarga de paquetes de `openstack` en primer lugar debemos de añadir los repositorios y hacer un *update y dist-upgrade* mediante `apt`. Como este proceso era muy lento computacionalmente hablando se hizo en la imagen base sobre la que se construyen las máquinas. Por ello mediante `apt` podemos instalar el paquete de `openstack python-openstackclient`.

Esta sería toda la configuración previa necesaria para el nodo de computación ya que el resto de servicios previos (como bases de datos, colas...) se instalaron en la máquina `controller`.

3.3.2- Compute Service (nova)

La base de este metaservicio es la misma que comentaba en la parte del controlador. Tiene las mismas API, servicios... En este momento lo que es importante mostrar es el proceso de instalación y configuración. El *hypervisor* que se va a usar es QEMU con una extensión KVM que permite la aceleración hardware de las instancias.

Los paquetes que debemos instalar son: `nova-compute` y `sysfsutils`.

Teniéndolos instalados debemos proceder a su configuración. Esta se encuentra en el fichero `nova.conf`. Las configuraciones las realizamos mediante *template* de `ansible` teniendo en cuenta los siguientes elementos:

- Configuración de `rabbitMQ` para las colas. Lo configuramos con las credenciales del usuario que creamos al principio.
- Seguimos posteriormente con las configuraciones de autenticación. Para ello indicamos que el sistema de autenticación será `keystone` y debemos de poner los datos de conexión para que el servicio pueda comunicarse.
- Modificamos la configuración de la parte de *networking* para que emplee `neutron`
- Habilitamos el acceso mediante VNC e introducimos los parámetros de conexión.
- Configuramos que el servicio de imágenes lo encuentre en el `controller` y activamos el *verbose* para obtener la máxima información posible.

Cuando todo esto se haya desplegado solo falta reiniciar el servicio `nova-compute` y debería de funcionar sin mayor problema.

Ahora que tenemos el servicio corriendo en ambos nodos (`controller` y `compute`) podemos seguir los pasos de verificación que nos indica la documentación oficial.

Ejecutando el comando '`nova service-list`' deberíamos ver cómo ya sale el servicio `nova-compute` como *enabled* y no como pasaba antes cuando teníamos solo el nodo controlador.

3.3.3- Networking Service (neutron)

El último servicio que queda por instalar en este nodo es `neutron`. Como el `controller`, por decirlo de algún modo, actúa como servidor, en este no son necesarias muchas de las cosas que en la máquina controladora sí lo eran.

Básicamente en este nodo hay que instalar los siguientes paquetes: `neutron-plugin-linuxbridge-agent` y `conntrack`.

Y las configuraciones que se realizan en el *template* son:

- Configuración del proveedor de colas y credenciales
- La autenticación con `keystone` el cual ha de ser referenciado así como tener las credenciales de acceso.
- Activar *verbose* para obtener la máxima información.

Al igual que ocurría en el `controller` debemos elegir si vamos a hacer uso de `provider-networks` o por el contrario usaremos `self-service networks`. En nuestro caso, como hemos configurado el `controller` para emplear `self-service networks` (privadas), debemos configurar esta máquina del mismo modo. Si hubiese más nodos se debería hacer igual.

Para ello lo primero que hay que configurar es el linux bridge agent mediante el archivo `template /etc/neutron/plugins/ml2/linuxbridge_agent.ini`. Configuramos cual será la interfaz pública, cual la ip interna y se activan los *security groups*.

Por último añadimos la configuración de `neutron` en el archivo de configuración de `nova` y reiniciamos los servicios de `nova` y `neutron`: `nova-compute`, `neutron-plugin-linuxbridge-agent`.

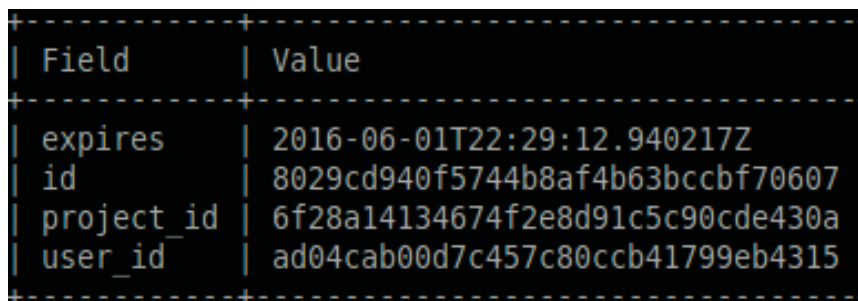
CAPÍTULO 4: PRUEBAS DEL SISTEMA

Para comprobar el estado real del desarrollo del proyecto se procedió a realizar pruebas de los metaservicios que forman openstack así como del conjunto global mediante **horizon**. Las pruebas de los metaservicios están expuestas en la documentación de la distribución de **openstack** de manera que, además de pruebas funcionales, podemos asegurar de manera certera que el sistema funcione.

Comenzamos realizando las pruebas de **keystone**, y para ello nos sugieren hacer una petición de token al servicio. La llevamos a cabo del siguiente modo:

```
ubuntu@controller:~$ openstack --os-auth-url http://controller:35357/v3 \  
> --os-project-domain-id default --os-user-domain-id default \  
> --os-project-name admin --os-username admin --os-auth-type password \  
> token issue
```

Esta petición nos pedirá introducir la contraseña del usuario, y si se autentica correctamente, nos devolverá un token con una fecha de expiración. Nos lo muestra por pantalla como se muestra en la Figura 22.



Field	Value
expires	2016-06-01T22:29:12.940217Z
id	8029cd940f5744b8af4b63bccbf70607
project_id	6f28a14134674f2e8d91c5c90cde430a
user_id	ad04cab00d7c457c80ccb41799eb4315

Figura 22: Token generado por keystone

El siguiente servicio al que se le realizaron las pruebas fue **glance**. En primer lugar debemos añadir las siguientes variables de entorno para poder efectuar las operaciones:

```
export OS_USERNAME=admin  
export OS_PASSWORD=a  
export OS_TENANT_NAME=admin  
export OS_AUTH_URL=http://controller:35357/v2.0
```

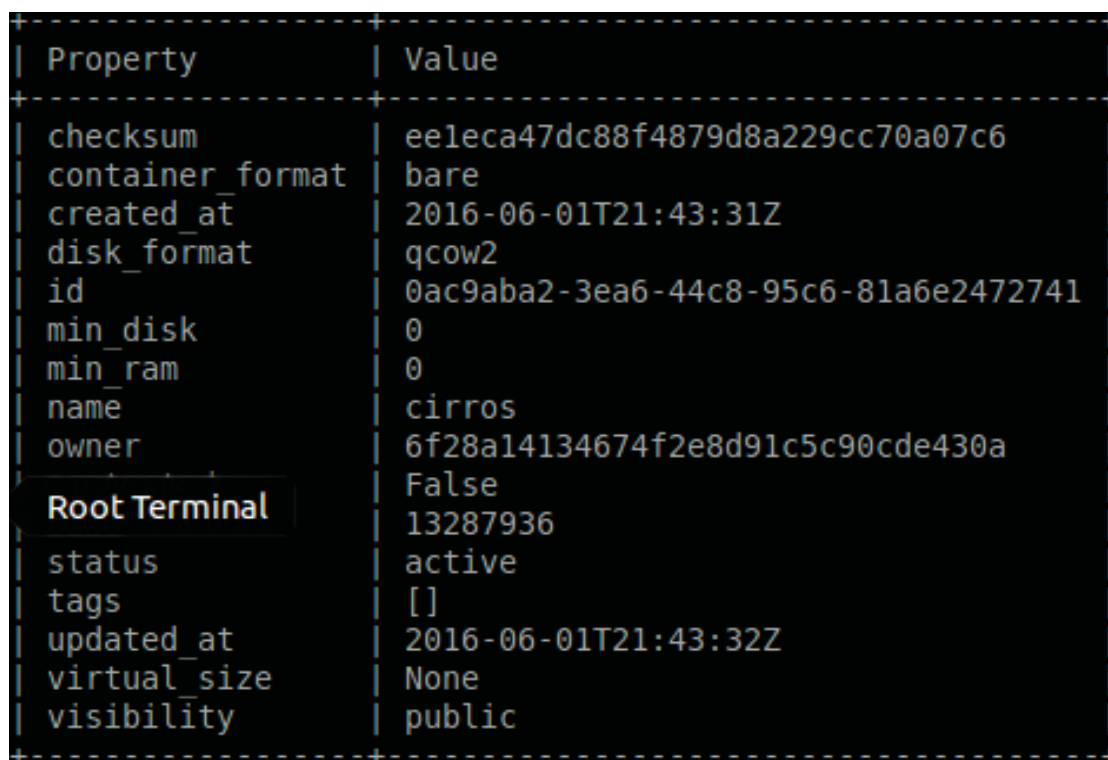
La operación que vamos a llevar a cabo es crear una nueva imagen y para ello lo primero que debemos hacer es descargarla desde la siguiente URL:

```
wget http://download.cirros-cloud.net/0.3.4/cirros-0.3.4-x86_64-disk.img
```

Cuando ya está en la máquina de control se ejecuta el siguiente comando de `openstack` para llevar a cabo la operación de crear la imagen:

```
glance image-create --name "cirros" \  
--file cirros-0.3.4-x86_64-disk.img \  
--disk-format qcow2 --container-format bare \  
--visibility public --progress
```

Esto produce una salida por pantalla con los datos que identifican la imagen creada. Debe ser similar a la de la Figura 23.



Property	Value
checksum	eeleca47dc88f4879d8a229cc70a07c6
container_format	bare
created_at	2016-06-01T21:43:31Z
disk_format	qcow2
id	0ac9aba2-3ea6-44c8-95c6-81a6e2472741
min_disk	0
min_ram	0
name	cirros
owner	6f28a14134674f2e8d91c5c90cde430a
protected	False
size	13287936
status	active
tags	[]
updated_at	2016-06-01T21:43:32Z
virtual_size	None
visibility	public

Figura 23: Imagen generada por `glance`

Ahora debemos realizar las pruebas en el servicio de computación (`nova`). Para ello como en el caso anterior debemos exportar las variables de entorno pertinentes:

```
export OS_USERNAME=admin
export OS_PASSWORD=a
export OS_TENANT_NAME=admin
export OS_AUTH_URL=http://controller:35357/v2.0
```

Una vez insertadas las variables el primer paso es listar los servicios para ver su estado como se muestra en la Figura 24.

```
ubuntu@controller:~$ nova service-list
```

Id	Binary	Host	Zone	Status	State	Updated_at	Disabled Reason
1	nova-cert	controller	internal	enabled	up	2016-06-01T22:01:02.000000	-
2	nova-conductor	controller	internal	enabled	up	2016-06-01T22:01:04.000000	-
3	nova-consoleauth	controller	internal	enabled	up	2016-06-01T22:01:02.000000	-
4	nova-scheduler	controller	internal	enabled	up	2016-06-01T22:01:02.000000	-
5	nova-compute	compute-VirtualBox	nova	enabled	up	2016-06-01T22:01:07.000000	-

Figura 24: Servicio de cómputo en funcionamiento.

Si el estado es *enabled* significa que el servicio está corriendo y funciona. Ahora podemos verificar si hay conexión con el servicio de autenticación listando los *endpoints* de las APIs como se muestra en la Figura 24.

glance	Value
adminURL	http://controller:9292
id	237734157fab45239388a2a696d6e359
internalURL	http://controller:9292
publicURL	http://controller:9292
region	RegionOne
nova	Value
adminURL	http://controller:8774/v2/6f28a14134674f2e8d91c5c90cde430a
id	213b3feeaf8641d28cfbb76b3e035a59
internalURL	http://controller:8774/v2/6f28a14134674f2e8d91c5c90cde430a
publicURL	http://controller:8774/v2/6f28a14134674f2e8d91c5c90cde430a
region	RegionOne
neutron	Value
adminURL	http://controller:9696
id	63aef9ec215f44ee90070fcf798d6d56
internalURL	http://controller:9696
publicURL	http://controller:9696
region	RegionOne
keystone	Value
adminURL	http://controller:35357/v2.0
id	1987ed935c4e47f0931e796e28ef0fdf
internalURL	http://controller:5000/v2.0
publicURL	http://controller:5000/v2.0
region	RegionOne

Figura 25: Endpoints de las APIs de los servicios.

Vemos que se muestran los endpoints de los servicios instalados correctamente.

El último servicio independiente que hay que probar es *neutron*. Para ello listamos sus agentes para comprobar en que estado están. La salida se muestra en la Figura 25.


```
ubuntu@controller:~$ neutron agent-list
```

id	agent_type	host	alive	admin_state_up	binary
16a9438d-07c8-4019-a438-52921def0e6a	DHCP agent	controller	:)	True	neutron-dhcp-agent
2415c5dd-3913-4aaa-8ba2-e234ba7c12e7	Linux bridge agent	compute-VirtualBox	:)	True	neutron-linuxbridge-agent
75bf9d7c-4178-4001-9cff-dabcead2ceb2	Metadata agent	controller	:)	True	neutron-metadata-agent
7b4b81c9-c899-48ab-bf6c-5c37224d7334	Linux bridge agent	controller	:)	True	neutron-linuxbridge-agent
f1bdf573-5549-4d87-bd52-bfb5f11beb71	L3 agent	controller	:)	True	neutron-l3-agent

Figura 26: Estado de los agentes de neutron

Se observa que todos están *alive* por lo que todo funciona correctamente.

Si queremos comprobar que la interfaz gráfica se adecúa a todos los servicios llevaremos a cabo un caso de uso concreto. En las siguientes figuras (26, 27, 28 y 29) se observa cómo creamos una instancia mediante el *dashboard*.

Figura 27: login de openstack

Proyecto

Compute

Vista general

Instancias

Imágenes

Acceso y seguridad

Red

Administrador

Identity

admin

admin

Instancias

Nombre de la instancia

Filter

Filtrar

Lanzar instancia

Terminar Instancias

More Actions

	Nombre de la instancia	Nombre de la imagen	Dirección IP	Tamaño	Par de claves	Estado	Zona de Disponibilidad	Tarea	Estado de energía	Tiempo desde su creación	Actions
☐	private-instance	cirros	10.10.11.3 IPs flotantes: 10.0.0.52	m1.tiny	mykey	Apagada	nova	Ninguno	Cerrar	1 semana	Iniciar Instancia

Displaying 1 item

127.0.0.1

Figura 28: Apartado de instancias del *dashboard*

Lanzar instancia

Detalles *

Acceso y seguridad

Redes *

Pos-creación

Opciones avanzadas

Zona de Disponibilidad

nova

Nombre de la instancia *

nueva

Sabor * ?

m1.tiny

Recuento de instancias * ?

1

Origen de arranque de la instancia * ?

Arrancar desde una imagen

Nombre de la imagen *

cirros (12,7 MB)

Especifique los detalles de la instancia a lanzar.

La siguiente tabla muestra los recursos utilizados por este proyecto en relación a sus cuotas.

Detalle del sabor

Nombre	m1.tiny
VCPU	1
Disco raíz	1 GB
Disco efímero	0 GB
Total de Disco	1 GB
RAM	512 MB

Límites del proyecto

Número de instancias

1 de 10 Usados

Número de VCPU

1 de 20 Usados

Total RAM

512 de 51.200 MB Usados

Cancelar

Lanzar

Figura 29: Configuración de una nueva instancia

Lanzar instancia

Detalles *

Acceso y seguridad

Redes *

Pos-creación

Opciones avanzadas

Redes seleccionadas

NIC:1 private (cf0c9c90-cc7c-45f6-96d5-3441b06f1d63)

Redes disponibles

public (512b76dd-a075-41db-ad51-dd342bf17a0e)

Escoger una red de "Redes disponibles" a "Redes seleccionadas" pulsando el botón o arrastrando y soltando. También puede cambiar el orden de la NIC arrastrando y soltando.

Cancelar

Lanzar

Figura 30: Configuración de redes de la instancia

Y tras pinchar en lanzar se crea la nueva máquina como se observa en la Figura 30.

ubuntu

admin

Proyecto

Compute

Vista general

Instancias

Imágenes

Acceso y seguridad

Red

Administrador

Identity

Instancias

Nombre de la instancia

Filter

Filtrar

Lanzar instancia

Terminar instancias

More Actions

Nombre de la instancia	Nombre de la imagen	Dirección IP	Tamaño	Par de claves	Estado	Zona de Disponibilidad	Tarea	Estado de energía	Tiempo desde su creación	Actions
nueva	cirros	10.10.11.4	m1.tiny	mykey	Activo	nova	Ninguno	Ejecutando	0 minutos	Crear Instancia
private-instance	cirros	10.10.11.3 IPs flotantes: 10.0.0.52	m1.tiny	mykey	Apagada	nova	Ninguno	Cerrar	1 semana	Iniciar Instancia

Displaying 2 items

127.0.0.1

Figura 31: Instancia creada

CAPÍTULO 5: RESULTADOS Y CONCLUSIONES

Tras este largo camino que ha sido para mi el proyecto los resultados obtenidos son significativos y bastante valiosos. Partiendo de una tecnología completamente desconocida como es `openstack` he sido capaz de crear mi propio *cloud* privado gracias a las tecnologías `opensource`. Pero este no es el mayor hito, sino que además lo hago de forma automática. Creo que es muy satisfactorio.

Gracias a esto puedo acceder a un portal web que me ofrece nada más entrar el *dashboard* de `openstack` para hacer login. Cuando introducimos las credenciales correctamente podemos acceder a todos y cada uno de los servicios. Quizá el elemento más importante parezca el levantar una instancia a la cual se le puede asignar además una IP flotante para acceder mediante ella. Pero no sólo podemos acceder a la máquina haciendo el tradicional `ssh`, sino que además, gracias a esta tecnología, podemos acceder mediante VNC a través de una interfaz web. Con el servicio de imágenes podremos realizar *backups* o simplemente guardar esa “foto” de la máquina para el futuro y es que, ¿dónde además de AWS, azure o google cloud engine podemos realizar estas funciones? `Openstack` es la respuesta. Al ser un conjunto de desarrollos independientes van por detrás en los avances pero hay que recordar que es gratuito. Simplemente hay que montarlo y con esta automatización creo que es más fácil.

Es cierto que a pesar de estar satisfecho con el esfuerzo realizado de cara al futuro estaría muy bien seguir desarrollando y mejorar algunos elementos. Creo que hay margen de mejora en cuanto a la idempotencia de *playbooks* y roles de `ansible` se refiere. Es posible que encontremos comandos que no cumplan ese principio y nos puedan conducir a fallos si hacemos varias veces el *deploy*. Además me gustaría probar con más nodos para separar aún más los servicios pero por las características de mi hardware no ha sido posible. De todos modos de cara al futuro estaría bien implementar la parte de *storage* de `openstack`.

En cuanto a la mejora se refiere se me ocurre un campo interesante hacia lo que trasladar esto, el uso de contenedores. Ya he conseguido que funcione la infraestructura desplegando con `ansible` pero aquí se encuentra el factor software de por medio. Se me ocurre, ¿por qué no? Crear contenedores de `docker` que se aprovisionen con `ansible` y de este modo abstraer la base sobre la que hay que construir el proyecto.

Por último en cuanto a mejora, y creo que la más importante (dejando `docker` a un lado), es instaurar `vagrant`[5] o algún elemento similar para la creación de las máquinas. Con el podría descargar la imagen y crear la máquina virtual dinámicamente sin depender de la imagen base de la que dependo ahora. Sería lo más apropiado, pero de eso me he dado cuenta más tarde. Por ello para una segunda fase del proyecto creo

que sería acertado hacer algo parecido.

Como conclusiones saco que la administración de este tipo de sistemas debe de ser una tarea bastante complicada sobre todo si pensamos en un entorno productivo. Por otra parte comparándolo con otro tipo de nubes a las que hay actualmente parece que van un poco por detrás con los servicios, pero como he comentado anteriormente, es un servicio gratuito. Creo que para un entorno productivo `openstack` sólo es rentable si es para una empresa grande con gran capital y hay datos privados que proteger.

BIBLIOGRAFÍA

Aquí expongo los enlaces que me han sido de utilidad en este trabajo y de donde he sacado la mayoría de la información que necesitaba:

- [1] <http://docs.openstack.org/liberty/>
- [2] <https://en.wikipedia.org/wiki/OpenStack>
- [3] <https://opensource.com/resources/what-is-openstack>
- [4] <https://en.wikipedia.org/wiki/VirtualBox>
- [5] <https://www.vagrantup.com/docs/virtualbox>
- [6] <https://help.ubuntu.com/community/VirtualBox>
- [7] <https://www.virtualbox.org/manual/UserManual.html>
- [8] <http://docs.ansible.com/ansible/index.html>
- [9] <https://galaxy.ansible.com/>

ANEXO 1: ACRÓNIMOS

IAAS: *Infrastructure as a service*. Modelo cloud en el que se ofrece servidores como servicio. El usuario debe proveer software, configuraciones...

SAAS: *Software as a service*. En este caso se ofrece el software como servicio. El usuario no se debe preocupar de la infraestructura, instalación... Sólo del uso.

PAAS: *Platform as a Service*. En este caso se ofrece la infraestructura ya gestionada así como las herramientas necesarias para el usuario preconfiguradas.

CPD: Centro de procesamiento de datos. Son edificios convenientemente adecuados para el *housing* de los servidores.

URI: (identificador de recursos uniforme) Cadena de caracteres que identifica un recurso REST.

QEMU: (*Quick Emulator*) Es un emulador de procesadores capaz de traducir dinámicamente binarios.

KVM: (*Kernel-based Virtual Machine*): Sistema de virtualización que permite correr máquinas virtuales con imágenes de sistemas operativos.

NTP: (*Network Time Protocol*): Protocolo para sincronizar sistemas informáticos.

AWS: (*Amazon web services*): *Cloud* público de pago con gran expansión.

ANEXO 2: GLOSARIO

Orquestador: Elemento software que permite la interacción entre sistemas aportando la lógica suficiente como para realizar operaciones de configuración, mantenimiento o instalación en ellos.

DHCP AGENT: Software encargado de asociar a las máquinas virtuales una IP de manera automática.

Playbook: En términos de Ansible, es un fichero yaml que contiene conjuntos de tareas para llevarse a cabo en alguna máquina.

Roles: En términos de Ansible, son todas las configuraciones necesarias para realizar las tareas en un tipo de máquina concreta. Por ejemplo, un rol *webserver* se encargaría de configurar un servidor web instalando y configurando todo lo necesario. Normalmente consta de *tasks* (tareas), *templates* (plantillas), *files* (ficheros), *handlers* (manejadores), *vars* (variables) y configuraciones por defecto y adicionales.

REST: *Representational State Transfer* o Transferencia de Estado Representacional. Es un estilo de arquitectura software para sistemas distribuidos. Se caracteriza por emplear protocolo sin estado, operaciones definidas (GET, POST, PUT y DELETE) y por tener los recursos identificados inequívocamente por una URI.

Este documento esta firmado por



Firmante	CN=tfgm.fi.upm.es, OU=CCFI, O=Facultad de Informatica - UPM, C=ES
Fecha/Hora	Sun Jun 05 13:12:24 CEST 2016
Emisor del Certificado	EMAILADDRESS=camanager@fi.upm.es, CN=CA Facultad de Informatica, O=Facultad de Informatica - UPM, C=ES
Numero de Serie	630
Metodo	urn:adobe.com:Adobe.PPKLite:adbe.pkcs7.sha1 (Adobe Signature)